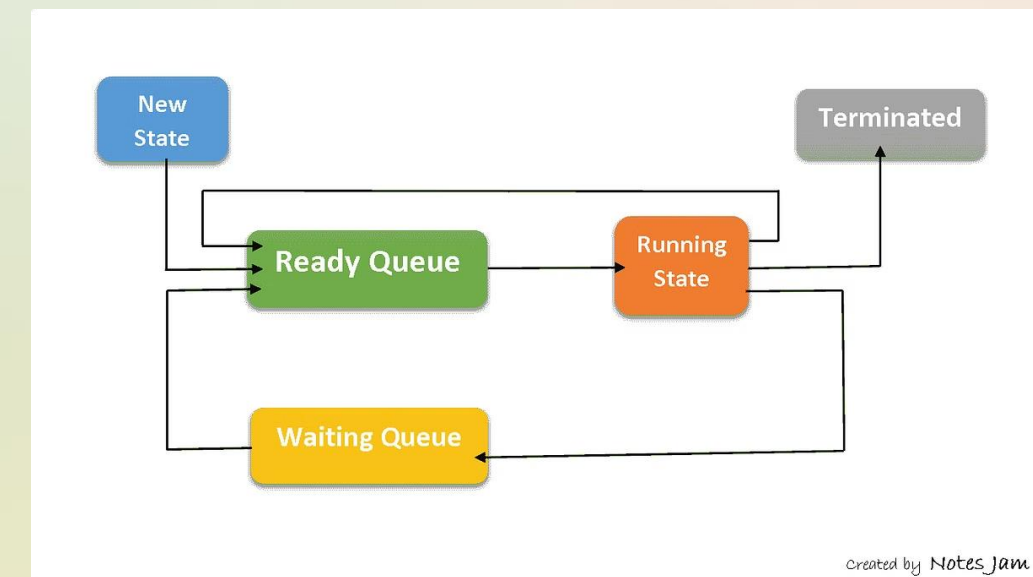


Programación de servicios y procesos



UT1. Programación multiproceso

Veremos los conceptos clave de la programación multiproceso y cómo los procesos interactúan con el sistema operativo. Aprenderemos sobre los estados de los procesos, la planificación del sistema operativo y la comunicación y sincronización entre procesos.





REFLEXIONA

- 1. ¿Sabes cuántos programas está ejecutando tu ordenador ahora mismo, sin que tú hayas abierto ninguno?**
- 2. ¿Sabes cómo tu ordenador ejecuta 200 programas a la vez si solo tiene 8 núcleos?**
- 3. ¿Sabes por qué Chrome abre un proceso distinto por cada pestaña en lugar de gestionarlo todo en uno?**
- 4. ¿Sabes que un programa tuyo puede lanzar otros programas, hablar con ellos y decidir cuándo cerrarlos?**

RESULTADO DE APRENDIZAJE: RA1

RA1.- Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.

CRITERIOS DE EVALUACIÓN

a)	Se han reconocido las características de la programación concurrente y sus ámbitos de aplicación.
b)	Se han identificado las diferencias entre programación paralela y programación distribuida, sus ventajas e inconvenientes
c)	Se han analizado las características de los procesos y de su ejecución por el sistema operativo
d)	Se han caracterizado los hilos de ejecución y descrito su relación con los procesos
e)	Se han utilizado clases para programar aplicaciones que crean subprocesos.
f)	Se han utilizado mecanismos para compartir información con los subprocesos iniciados.
g)	Se han utilizado mecanismos para sincronizar y obtener el valor devuelto por los subprocesos iniciados.
h)	Se han desarrollado aplicaciones que gestionen y utilicen procesos para la ejecución de varias tareas en paralelo.
i)	Se han depurado y documentado las aplicaciones desarrolladas.



Índice



1

Introducción a la programación multiproceso

- 1.1 Programa, ejecutable, proceso y servicio
- 1.2 Multitarea: concurrencia y paralelismo.
- 1.3 Programación concurrente: características y ámbitos
- 1.4 Programación paralela y distribuida

2

El proceso y el sistema operativo

- 2.1 Principales estados de un proceso y cambios de estado
- 2.2 Bloque de control de procesos (PCB) y colas de procesos
- 2.3 Planificación de procesos (cargador, planificadores y algoritmos)
- 2.4 Cambio de contexto

3

Gestión de procesos con herramientas del SO

- 3.1 Administrador de tareas
- 3.2 Comandos en Windows: tasklist / taskkill
- 3.3 Equivalentes en GNU/Linux

Índice



4 Hilos frente a procesos

4.1 Qué es un hilo

4.2 Cuándo conviene un proceso y cuándo un hilo

5 Creación y gestión de procesos desde Java

5.1 Creación de procesos

5.1.1 Las clases Process y ProcessBuilder

5.1.2 Argumentos, directorio de trabajo y entorno

5.1.3 Lanzamiento y ciclo de vida del proceso hijo

5.2 Comunicación entre procesos

5.2.1 Entrada, salida y error estándar

5.2.2 Los flujos del hijo vistos desde el padre

5.2.3 Redirecciones y encadenamiento de procesos

5.3 Sincronización entre procesos

5.3.1 Espera del proceso hijo: `waitFor()`

5.3.2 Códigos de salida

5.3.3 Espera asíncrona: `onExit()`

5.3.4 Consulta y terminación: `ProcessHandle` y `destroy()`

Índice



6

Aplicaciones multiproceso en paralelo

6.1

Reparto de una tarea entre varios subprocesos

6.2

Medición frente a la ejecución secuencial

7

Documentación y depuración

8

Bibliografía

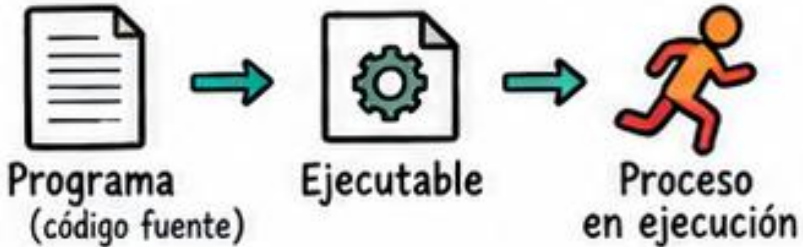
1. INTRODUCCIÓN

- Programa, ejecutable, proceso y servicio
- Multitarea: concurrencia y paralelismo
- Características y ámbitos de la programación concurrente
- Programación paralela y distribuida



IDEA CLAVE

Idea clave:
un proceso es
un programa
en ejecución



CONCURRENCIA

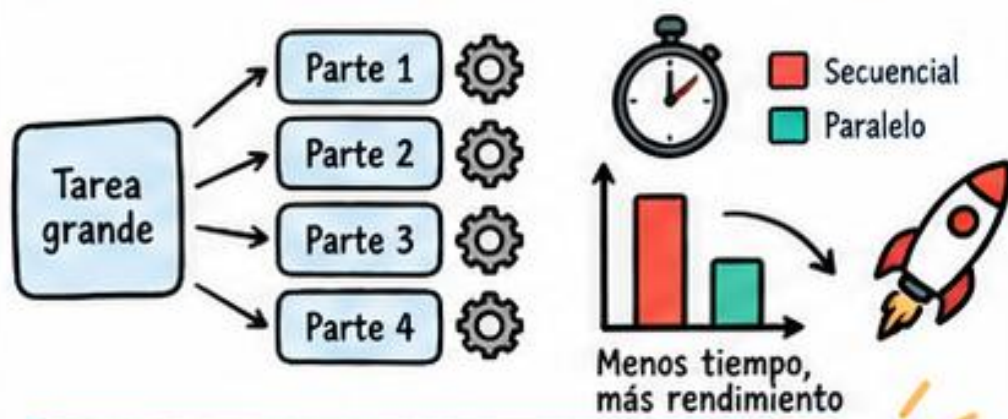


PARALELISMO



6. APLICACIONES MULTIPROCESO EN PARALELO

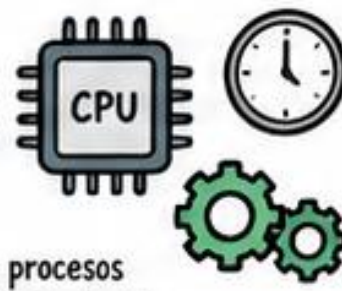
- Reparto de una tarea entre varios subprocesos
- Lanzar primero y esperar después
- Medición frente a la ejecución secuencial
- Aceleración = tiempo secuencial ÷ tiempo paralelo



⚠ Paralelizar no siempre acelera:
primero hay que medir

2. EL PROCESO Y EL SISTEMA OPERATIVO

- Estados de un proceso y cambios de estado
- PCB y colas de procesos
- Planificación: cargador, planificadores y algoritmos
- Cambio de contexto



El sistema operativo
decide quién usa la CPU
y cuándo

3. GESTIÓN DE PROCESOS CON HERRAMIENTAS DEL SISTEMA

- Administrador de tareas
- Windows: tasklist y taskkill
- GNU/Linux: equivalentes para ver y terminar procesos



Administrador de tareas			
Proceso	PID	CPU	Memoria
java.exe	1234	2%	120 MB
chrome.exe	5678	1%	250 MB
explorer.exe	9012	0%	80 MB
...	...	...	...

```
C:\> tasklist
C:\> taskkill /PID 1234

$ ps -ef
$ kill 1234
```

Primero observamos y gestionamos procesos desde el sistema

UT1 · PROGRAMACIÓN MULTIPROCESO

Del concepto de proceso al control desde Java y la ejecución en paralelo

5. CREACIÓN Y GESTIÓN DE PROCESOS DESDE JAVA

5.1 Creación de procesos

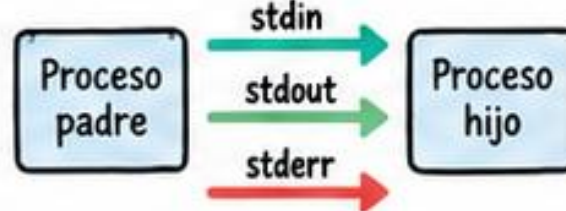
- Process y ProcessBuilder
- Argumentos, directorio de trabajo y entorno
- Lanzamiento y ciclo de vida del proceso hijo



Aquí pasamos del sistema operativo al código Java

5.2 Comunicación entre procesos

- Entrada, salida y error estándar
- Los flujos del hijo vistos desde el padre
- Redirecciones y encadenamiento de procesos



5.3 Sincronización entre procesos

- waitFor()
- Códigos de salida
- onExit()
- ProcessHandle y destroy()



Crear → Comunicar → Esperar → Terminar

4. HILOS FRENTA A PROCESOS

- Qué es un hilo
- Cuándo conviene un proceso
- Cuándo conviene un hilo



7. DOCUMENTACIÓN Y DEPURACIÓN

- Explicar lo que hace la aplicación
- Comprobar resultados y errores
- Dejar evidencias, pruebas y conclusiones



Un buen programa también se entiende y se verifica

ENTENDER

OBSERVAR

COMPARAR

PROGRAMAR

COMUNICAR

SINCRONIZAR

PARALELIZAR

DEPURAR

UNA UNIDAD, MUCHAS CAPACIDADES

¡A POR EL SIGUIENTE
RETO!

1. Introducción a la programación multiproceso

1.1. Programa, ejecutable, proceso y servicio

Piezas distintas de un mismo puzle

Conceptos fundamentales

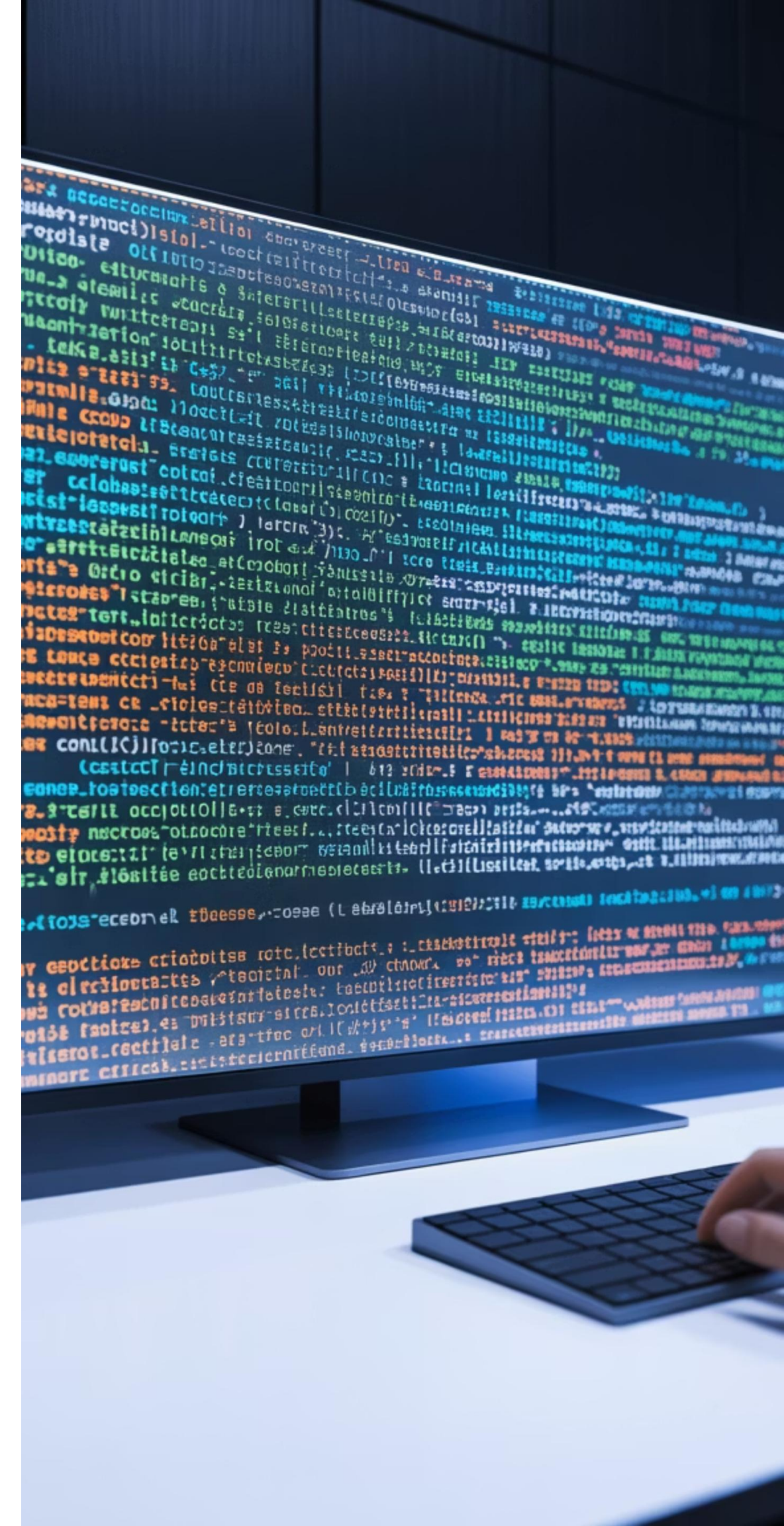
Para dominar la programación multiproceso, primero debemos tener muy claras las diferencias entre estos cuatro términos.

Relación íntima

Están íntimamente relacionados, pero no son lo mismo.

El gran coordinador

El **Sistema Operativo** es el gran coordinador que gestiona todos estos elementos. Es a él a quien le pedimos que ejecute los programas.



1.1. Programa, ejecutable, proceso y servicio

El viaje desde la idea hasta la ejecución

</>

El PROGRAMA (Código fuente)

El programador escribe el código en un editor de texto.
Son las instrucciones, la "receta".



El EJECUTABLE

El **compilador** traduce el código fuente a un lenguaje que el Sistema Operativo entiende. **Un archivo en disco.**
Es el "libro de recetas" listo para ser usado.

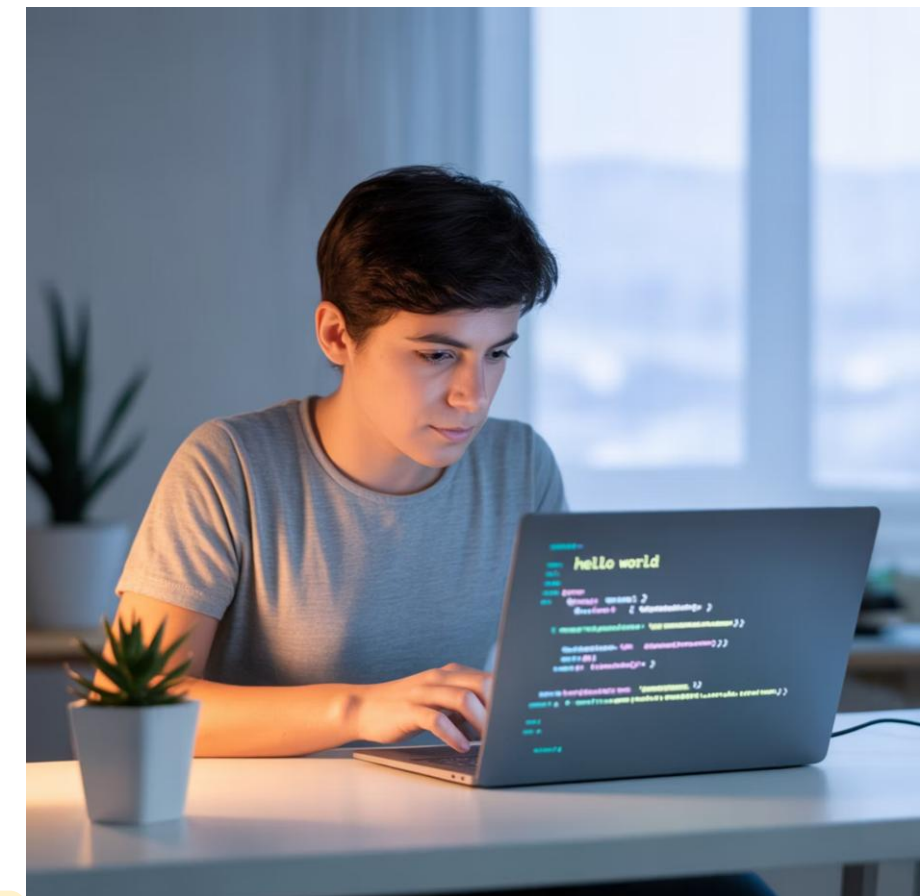


El PROCESO

El usuario hace doble clic en el ejecutable. En ese momento, el programa cobra vida y se convierte en un proceso.
chrome.exe es un único archivo. Si lo abres tres veces, tienes tres procesos independientes.
Es el "cocinero" siguiendo la receta.



Proceso = programa en ejecución. Es la definición que hay que llevarse de esta unidad.

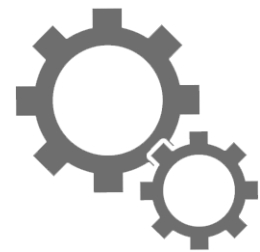


1.1. Programa, ejecutable, proceso y servicio

Un caso especial: los Servicios

Servicios: los trabajadores silenciosos del sistema

Un **servicio** también es un programa en ejecución, pero con unas características muy particulares:



Segundo plano

Se ejecuta en **segundo plano** (no ves una ventana).



Sin interacción

No requiere la interacción del usuario para funcionar.



Inicio automático

Normalmente, **se inicia de forma automática** con el sistema operativo y está siempre activo.

Ejemplos:



Antivirus

Protección continua del sistema



Actualizaciones

Servicio de actualización de Windows



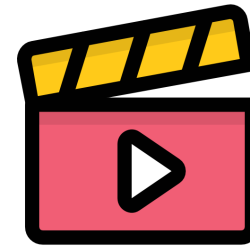
Base de Datos

Servidor de base de datos

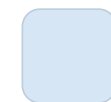


En Windows los ves en services.msc. En Linux se llaman demonios (daemon).

1.2. Multitarea: concurrencia y paralelismo



Video introducción (Aula Virtual)



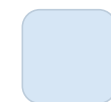
Tu ordenador:

Piensa en todo lo que tu ordenador hace ahora mismo: escuchas música, el antivirus está activo, descargas un archivo, te llegan notificaciones...



La pregunta clave

¿Cómo puede un ordenador ejecutar a la vez más tareas que el número de procesadores (núcleos) que tiene?



La respuesta

La respuesta tiene un nombre: **Multitarea**.

Definición: La multitarea es la capacidad de un computador de ejecutar varias tareas (programas) simultáneamente.

1.2. Multitarea: concurrencia y paralelismo

Multitarea real vs. Multitarea simulada

No toda la multitarea es igual. Depende del hardware disponible.

Multitarea REAL (Paralelismo)

- Ocurre cuando hay tantas (o más) unidades de proceso como tareas
- Cada tarea se ejecuta de verdad en su propio núcleo al mismo tiempo
- Es la situación ideal

Multitarea SIMULADA (Concurrencia)

- Ocurre cuando hay menos unidades de proceso que tareas
- El sistema operativo cambia entre tareas a una velocidad tan alta
- **Nos da la sensación** de que se ejecutan a la vez



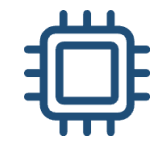
El hardware pone los núcleos; el sistema operativo decide quién entra y cuándo.
Toda ejecución paralela es concurrente. No toda ejecución concurrente es paralela.



1.2. Multitarea: concurrencia y paralelismo

¿Quién hace posible la multitarea? CPU + SO

La multitarea depende de una colaboración perfecta entre hardware y software.



El Procesador (CPU)

Aporta la **capacidad física**. Son los "músculos".



El Sistema Operativo (SO)

Aporta la **capacidad lógica**. Es el "cerebro" que organiza todo.



Conclusión clave:

Un procesador simple con un SO inteligente **PUEDE** tener multitarea. Un procesador de última generación con un SO inadecuado **NO** la aprovechará.

1.3. Programación concurrente: características y ámbitos

¿Qué aporta la programación concurrente?

1

Aprovechamiento del procesador

Mientras un proceso espera al disco o a la red, otro usa la CPU. Sin concurrencia, ese tiempo se pierde.

2

Capacidad de respuesta

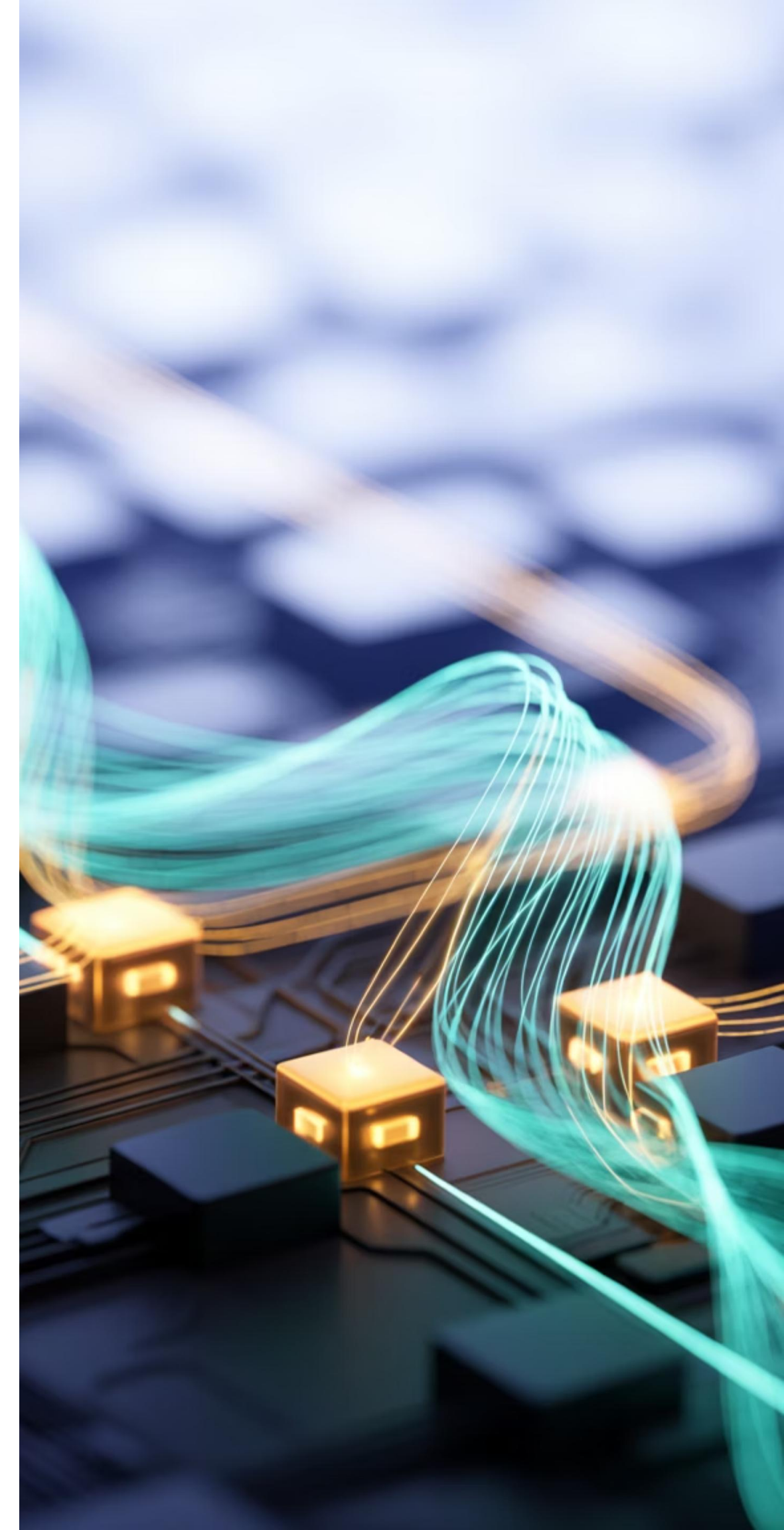
La interfaz sigue atendiéndote mientras el programa trabaja por debajo. Es la diferencia entre una aplicación viva y una congelada.

3

Escalabilidad

El trabajo se puede repartir entre más núcleos o más máquinas según haga falta.

 **El precio:** coordinar varias tareas es mucho más difícil que programar una sola. Todo lo que viene después de esta unidad va de eso.



1.3. Programación concurrente: características y ámbitos

Dónde se usa: ámbitos de aplicación

Servidores

Un servidor web atiende a miles de clientes a la vez. Sin concurrencia, atendería a uno y los demás esperarían.

Cálculo intensivo

Compilar un proyecto grande, renderizar vídeo, aplicar filtros a imágenes, simulaciones científicas: el trabajo se trocea y se reparte.

Aplicaciones interactivas

Videojuegos, editores, navegadores. La interfaz por un lado y el trabajo pesado por otro.

El propio sistema operativo

Es el primer programa concurrente de la máquina: sin él, nada de lo anterior sería posible.



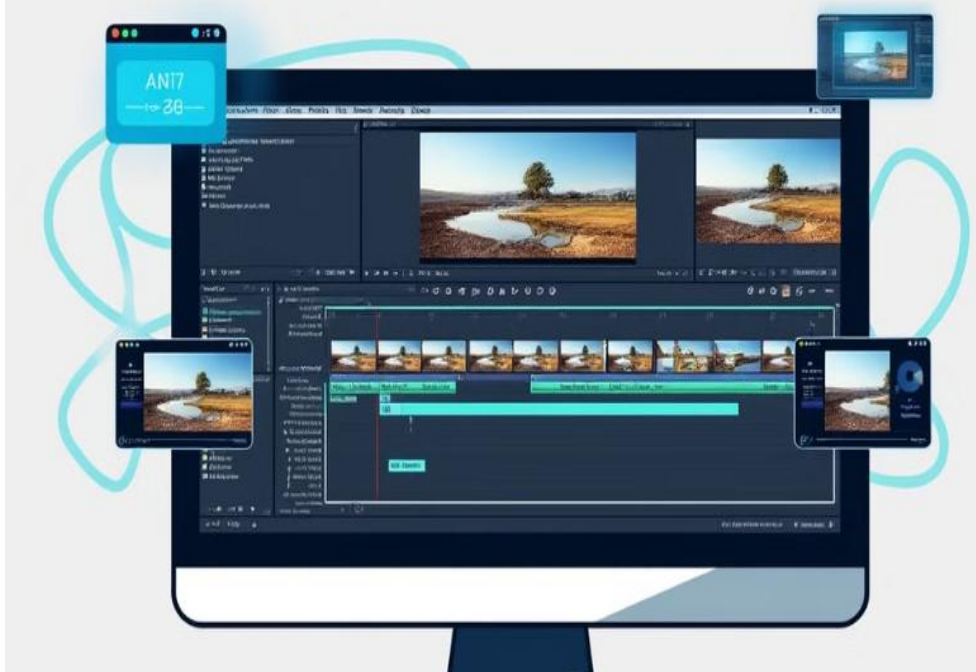
Compiled

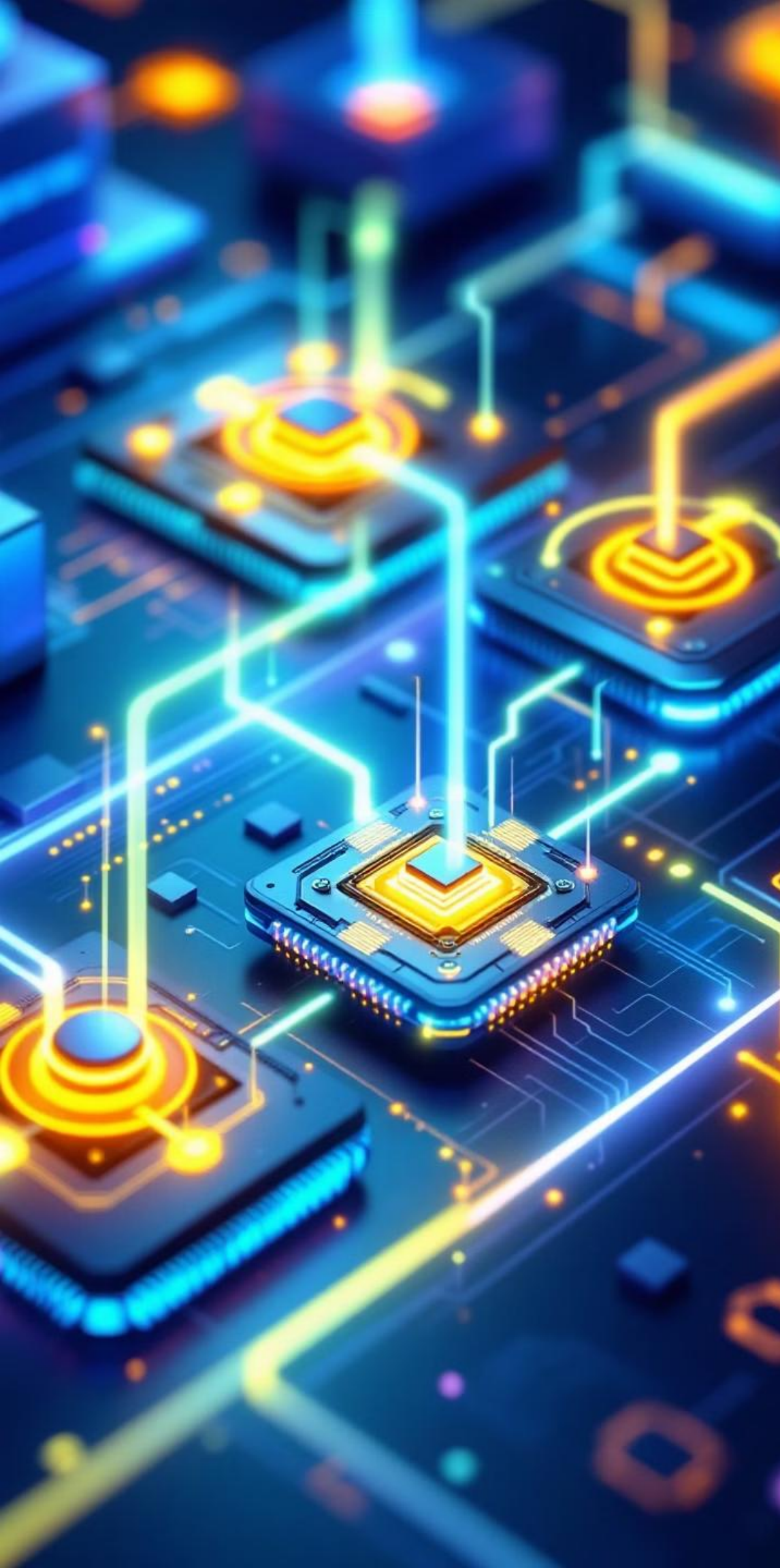


Compiled

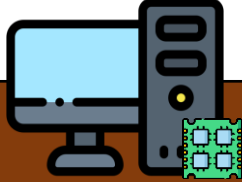


Ratay onhnorez "lippical -earelllel n ahdr, radeo yono lifotmdtion"





1.4. Programación paralela y distribuida

	 Paralela	 Distribuida
Dónde	Varios núcleos, una máquina	Varias máquinas en red
Memoria	Compartida	Cada una la suya
Se comunican	Leyendo la misma memoria	Enviándose mensajes

VENTAJAS	
Paralela	Distribuida
Comunicación inmediata y programación más sencilla.	Crece añadiendo máquinas y sobrevive a la caída de una de ellas.

INCONVENIENTES	
Paralela	Distribuida
Tienes el límite de núcleos del equipo, y compartir memoria abre la puerta a errores de acceso simultáneo.	La red añade retardo, puede fallar a mitad de una operación y depurar se complica mucho.

Un proceso, muchas máquinas o muchos núcleos. Pero ...
¿qué es exactamente un proceso para el sistema operativo? → 2

2. El proceso y el sistema operativo

2.1. Principales estados de un proceso

Un proceso no está siempre ejecutándose. Casi nunca lo está.

Un proceso, desde su creación hasta su finalización, pasa por varios estados controlados por el sistema operativo, los cuales optimizan el uso de recursos, especialmente el procesador.



- ☐ **Nuevo**

El proceso ha sido creado, pero aún no está listo para ejecución.

- ☐ **Listo**

El proceso está preparado para ejecutarse, esperando la asignación del procesador.

- ☐ **En ejecución**

El proceso tiene control del procesador y está realizando sus tareas.

- ☐ **Bloqueado**

El proceso está a la espera de un evento (como una operación de E/S) y no puede avanzar.

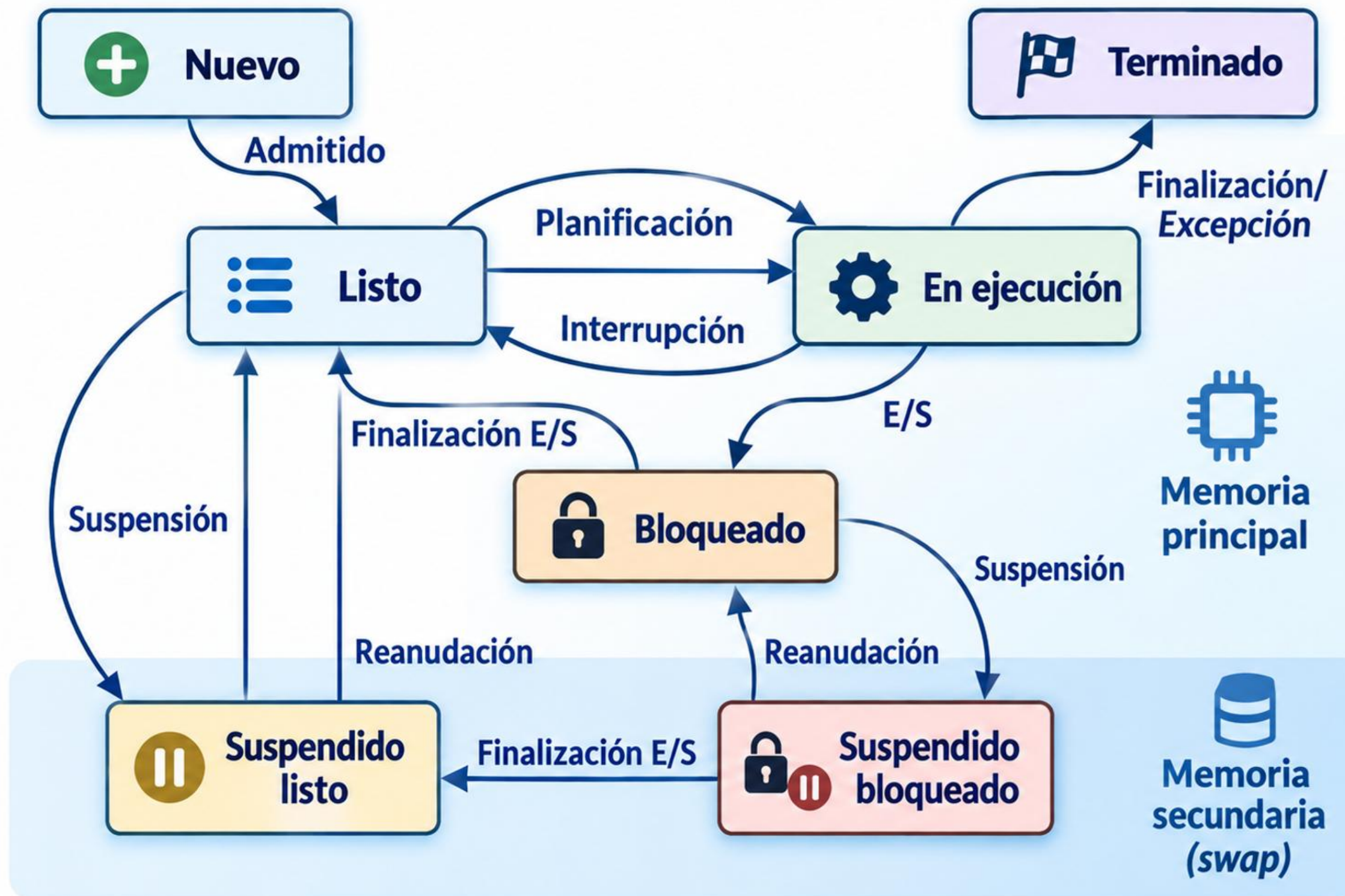
- ☐ **Suspendido**

El proceso se almacena temporalmente en la memoria secundaria por falta de recursos o espera de un evento.

- ☐ **Terminado**

El proceso ha concluido su ejecución y liberado sus recursos.

2.1. Cambios de estado



¿Quién provoca cada cambio y por qué?

Admisión: Nuevo → Listo.
El sistema decide que ya hay sitio en memoria.

Despacho: Listo → En ejecución.
El planificador le asigna el procesador.

Expulsión: En ejecución → Listo.
Se le acaba el tiempo asignado o llega algo más prioritario.

Bloqueo: En ejecución → Bloqueado.
El propio proceso pide una operación de entrada/salida.

Desbloqueo: Bloqueado → Listo.
Termina la operación que esperaba. Ojo: vuelve a Listo, no a ejecución.

De Bloqueado nunca se pasa directamente a En ejecución. Hay que volver a hacer cola.

2.2. Bloque de control de procesos (PCB) y colas de procesos



Bloque de control de procesos (PCB)

El sistema operativo necesita recordar todo lo de un proceso mientras no se ejecuta. Eso lo guarda en el Bloque de Control de Proceso, uno por cada proceso vivo.

¿Qué contiene?

1

Identificador (PID) y estado actual

2

Contador del programa

Indica en qué parte del código se está ejecutando el proceso.

3

Estado del procesador

Refleja el valor de los registros en el momento de la interrupción.

4

Imagen de memoria

El espacio que el proceso está utilizando en la memoria principal.

5

Recursos abiertos

Archivos, dispositivos, prioridad.

Sin PCB no habría multitarea: sería imposible retomar un proceso donde se quedó.

2.2. Bloque de control de procesos (PCB) y colas de procesos



Colas de procesos

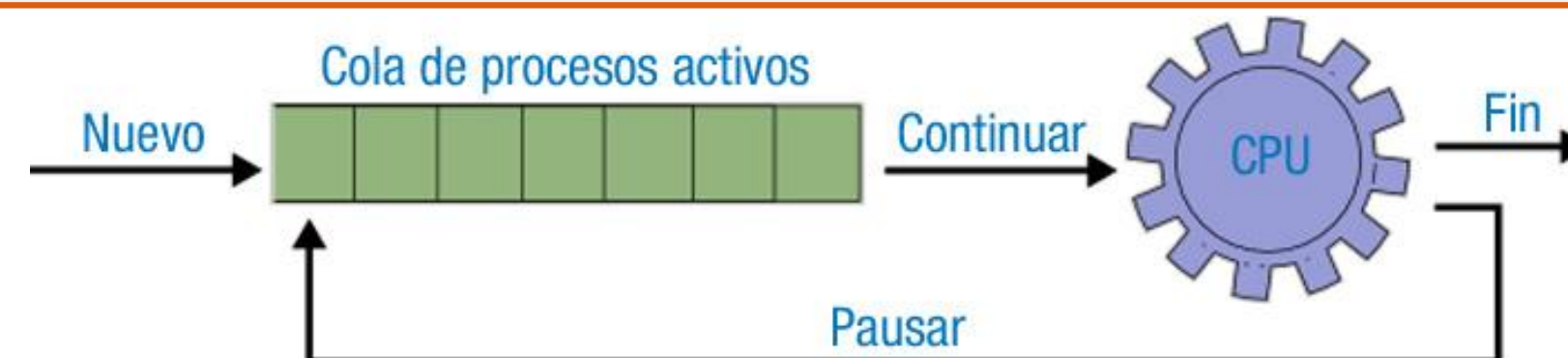
Los procesos se organizan en colas según lo que estén esperando.

COLAS DE PROCESOS LISTOS

- 1 Contiene todos los procesos que podrían ejecutarse ya. De aquí saca el planificador al siguiente proceso.

COLA DE PROCESOS BLOQUEADOS

- 2 Una cola por cada dispositivo: una para el disco, otra para la red, otra para el teclado. Cuando el dispositivo responde, el proceso sale de su cola y pasa a la de listos.



Cómo se implementan: Cada cola es en realidad una lista de PCB enlazados. Mover un proceso de estado es mover su PCB de cola.



2.3. Planificación de procesos (cargador)

Antes de que un proceso pueda competir por el procesador, alguien tiene que crearlo.

¿Qué hace el cargador?

- Reserva espacio en memoria principal para el proceso.
- Copia ahí las instrucciones, los datos y la pila.
- Crea el PCB con el PID y el estado inicial.
- Lo mete en la cola de listos.

A partir de ahí, ya no es asunto suyo

El cargador no decide cuándo se ejecuta: eso es del planificador.



Al hacer doble clic en un icono: todo lo que ocurre antes de que veas la ventana, es lo que hace el cargador.

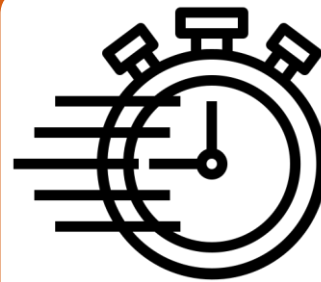


2.3. Planificación de procesos (planificador)



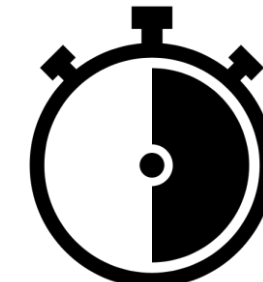
Planificador a largo plazo

Selecciona procesos para ser admitidos en la memoria principal, llevándolos al estado listo.



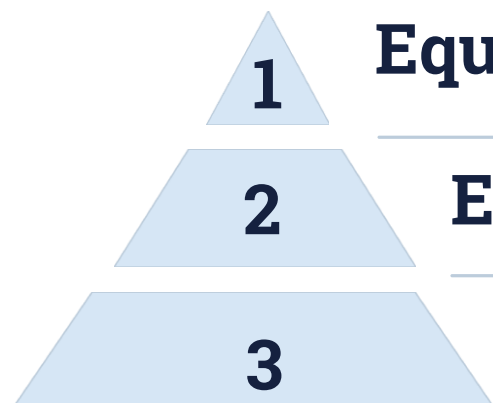
Planificador a corto plazo

Asigna el procesador a los procesos listos, realizando cambios de estado entre listo y en ejecución, basándose en interrupciones del sistema.



Planificador a medio plazo

Gestiona el intercambio (swap) entre memoria principal y secundaria, suspendiendo o reanudando procesos según la disponibilidad de recursos.



1 Equidad: que nadie se quede sin ejecutar

2 Eficiencia: que el procesador no esté parado

3 Tiempo de respuesta: que el usuario no note la espera.

Objetivos del planificador

Asigna el procesador a los procesos listos, realizando cambios de estado entre listo y en ejecución, basándose en interrupciones del sistema.

2.3. Planificación de procesos (Algoritmos)

El planificador es el encargado de decidir qué proceso se ejecutará y durante cuánto tiempo, utilizando distintos algoritmos de planificación, como:

Round-Robin

Asigna un quantum o porción de tiempo a cada proceso. Si el proceso no termina en ese tiempo, vuelve al final de la cola de listos. Este método es especialmente útil para sistemas interactivos.

Por prioridad

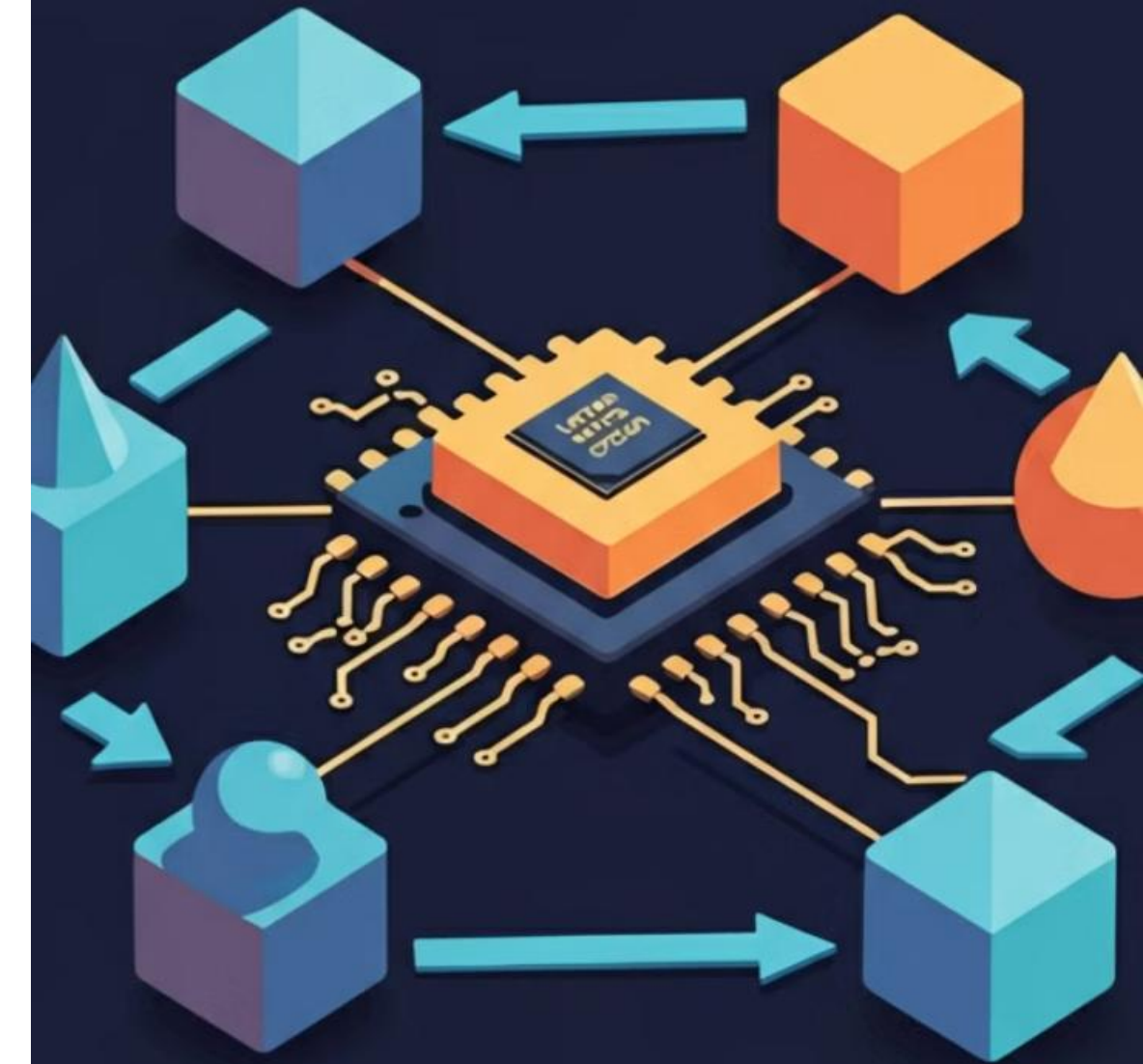
Cada proceso tiene una prioridad, y el sistema asigna el procesador a los procesos de mayor prioridad primero. Ideal para procesos críticos que no deben esperar.

Múltiples colas

Combina Round-Robin y Prioridad. Los procesos se organizan en colas según su prioridad, y cada cola es gestionada por el método Round-Robin.



Un proceso de prioridad baja puede no ejecutarse nunca si no dejan de llegar otros más urgentes. Se llama **inanición.**





2.4. Cambio de contexto

Es lo que ocurre cada vez que el procesador deja un proceso y coge otro.

Guardado del estado de la CPU:

1

El sistema operativo almacena en el Bloque de Control de Proceso (PCB) el estado del proceso actual, incluyendo el Contador de Programa (que indica la siguiente instrucción a ejecutar) y el puntero a pila (que apunta al contexto del proceso en la pila).

Restauración del estado del proceso entrante:

2

Luego, se cargan los valores de estos registros para el proceso entrante, permitiendo que continúe ejecutándose desde el punto donde quedó.

Todo esto no es gratis: Durante el cambio, el procesador no hace trabajo útil. Miles de cambios por segundo cuestan rendimiento.



Cambiar de proceso es caro porque hay que cambiar de espacio de memoria. Pero, **¿existe algo más barato dentro del mismo proceso?** → **Los hilos (apartado 4)**

3. Gestión de procesos con herramientas del sistema

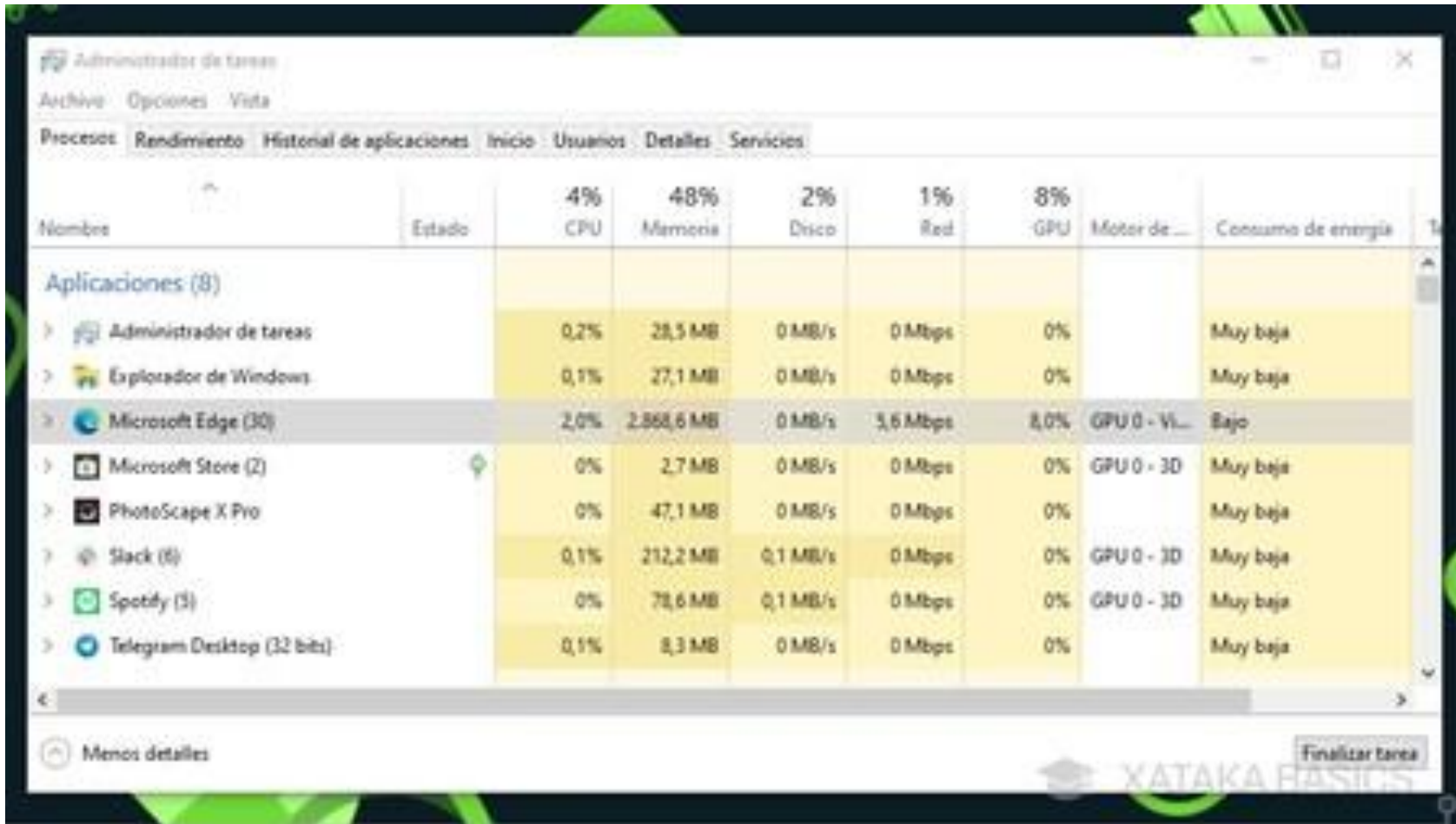
3.1. Administrador de tareas

Windows

La teoría del apartado 2 ahora por pantalla

El **Administrador de tareas** ofrece una interfaz intuitiva para ver y controlar los procesos activos.

Ctrl + Mayus + Esc (abre el administrador)



Nombre	Estado	4%	48%	2%	1%	8%	Motor de...	Consumo de energía
		CPU	Memoria	Disco	Red	GPU		
Aplicaciones (8)								
Administrador de tareas		0,2%	28,5 MB	0 MB/s	0 Mbps	0%		Muy baja
Explorador de Windows		0,1%	27,1 MB	0 MB/s	0 Mbps	0%		Muy baja
Microsoft Edge (30)		2,0%	2.868,6 MB	0 MB/s	5,6 Mbps	8,0%	GPU 0 - VL...	Bajo
Microsoft Store (2)		0%	2,7 MB	0 MB/s	0 Mbps	0%	GPU 0 - 3D	Muy baja
PhotoScape X Pro		0%	47,1 MB	0 MB/s	0 Mbps	0%		Muy baja
Slack (6)		0,1%	212,2 MB	0,1 MB/s	0 Mbps	0%	GPU 0 - 3D	Muy baja
Spotify (3)		0%	78,6 MB	0,1 MB/s	0 Mbps	0%	GPU 0 - 3D	Muy baja
Telegram Desktop (32 bits)		0,1%	8,3 MB	0 MB/s	0 Mbps	0%		Muy baja

Pestaña Procesos

Vista agrupada por aplicación. Consumo de CPU, memoria, disco y red en tiempo real.

Pestaña Detalles

La vista que nos interesa: un proceso por fila, con su PID, el usuario que lo lanzó y su estado.

¿Qué Podemos hacer desde aquí? Finalizar un proceso || cambiar su prioridad || ver el árbol de procesos hijos.

3. Gestión de procesos con heramientas del sistema

3.2. Comandos en Windows

Los sistemas operativos ofrecen varios comandos de terminal para la gestión de procesos. Estos son útiles para administradores y desarrolladores que buscan un control directo y detallado.

tasklist

Muestra una lista de procesos activos, incluyendo su PID y uso de memoria.

taskkill /PID

Finaliza el proceso con el identificador especificado, permitiendo un control rápido y específico.



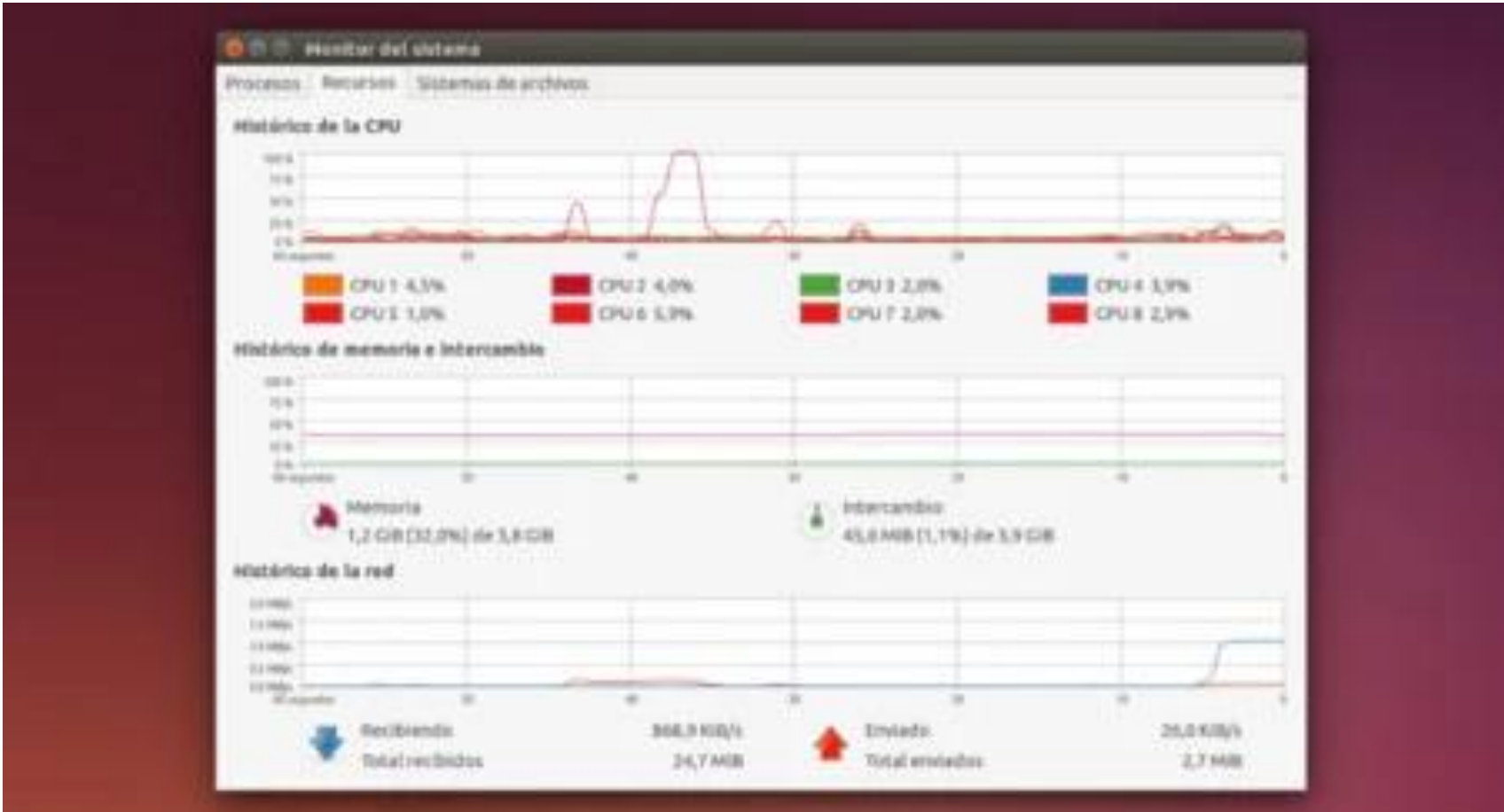
¿POR QUÉ APRENDER ESTO SI ESTÁ EL ADMINISTRADOR DE TAREAS?
Porque desde Java se lanzan comandos, no ventanas.

3. Gestión de procesos con heramientas del sistema

3.3. Equivalentes en GNU/Linux

Los conceptos son los mismos

PID, estado, prioridad y jerarquía padre-hijo existen igual en los dos sistemas. Cambia el nombre del comando.



Windows	GNU/Linux	Qué hace
tasklist	ps aux	Lista todos los procesos del sistema
—	pstree	Muestra los procesos como árbol de padres e hijos
taskkill /PID	kill PID	Termina un proceso por su identificador
Prioridad base	nice -n	Lanza un proceso con una prioridad concreta

No los vamos a usar en clase porque trabajaremos con Windows, pero los verás en cualquier servidor.



4. Hilos frente a procesos

4.1. Qué es un hilo

Un proceso puede hacer varias cosas a la vez por dentro.

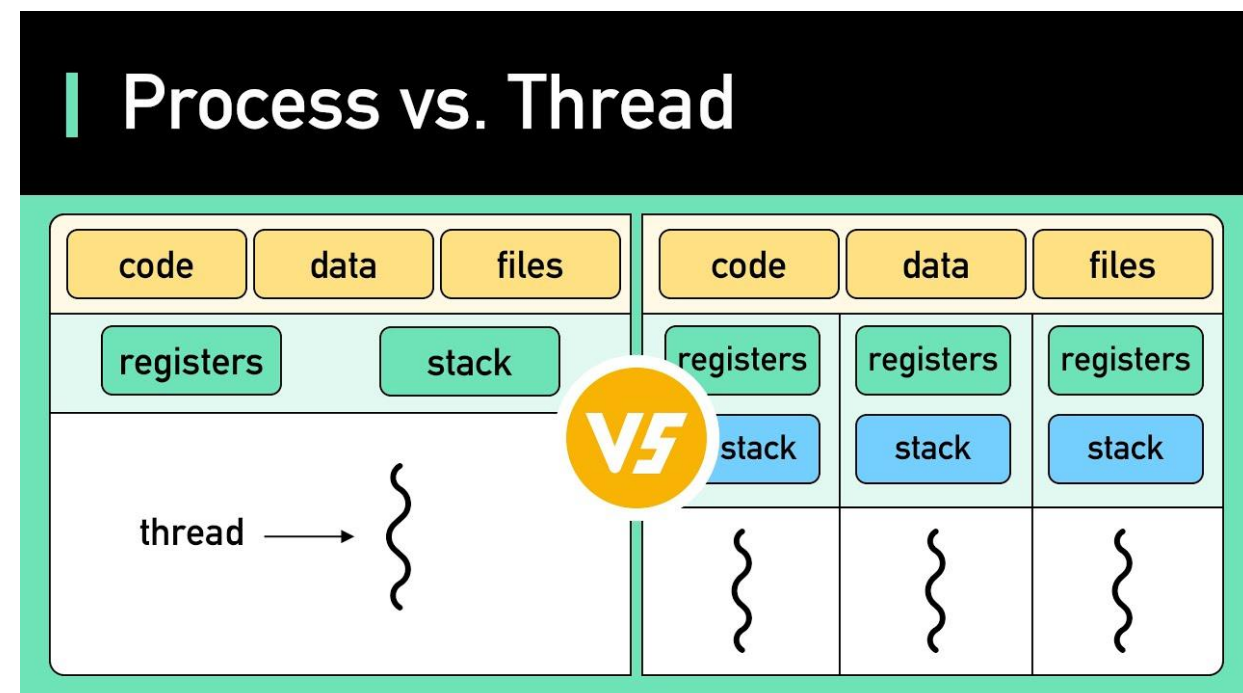
- Cada una de esas líneas de ejecución es un hilo.
- Un proceso tiene siempre al menos uno.
- Los hilos (o threads) representan la **unidad básica de ejecución** dentro de un proceso y se conocen también como **procesos ligeros**.

¿Qué comparten los hilos del mismo proceso?

- La memoria: las variables, los objetos, los datos.
- Los archivos y conexiones abiertos.

¿Qué tiene cada hilo para sí?

- Su propio contador de programa: por qué instrucción va
- Su propia pila: variables locales y llamadas a métodos.



[Enlace a video descriptivo](#)

4.1. Qué es un hilo



Crear un proceso es caro

Hay que reservar un espacio de memoria nuevo, copiar el código y los datos, y construir un PCB entero.



Crear un hilo es barato

El espacio de memoria ya existe: solo hay que darle una pila y un contador de programa.



Cambiar de hilo también es más barato

El cambio de contexto entre dos hilos del mismo proceso no cambia el espacio de memoria. Se ahorra la parte más costosa de lo que vimos en el apartado 2.8.

Pero para el planificador, sin embargo, son lo mismo

Reparte tiempo de procesador entre hilos igual que entre procesos: los mismos estados, las mismas colas.



La ventaja de los hilos no está en cómo se planifican, sino en lo que cuesta crearlos y alternarlos.

4.2. Cuando conviene un proceso y cuando un hilo



Ejemplo: el videojuego

- Los enemigos, la física, el sonido y la partida comparten el mapa, la puntuación y las vidas. Son **HILOS** del mismo proceso: necesitan ver los mismos datos al instante.
- El navegador que tienes abierto detrás es otro **PROCESO**: no comparte nada con el juego, ni debe.

	Hilos	Procesos
Memoria	Compartida	Separada
Si uno falla	Cae todo el proceso	Los demás siguen
Compartir datos	Inmediato	Hay que comunicarlos

La regla práctica

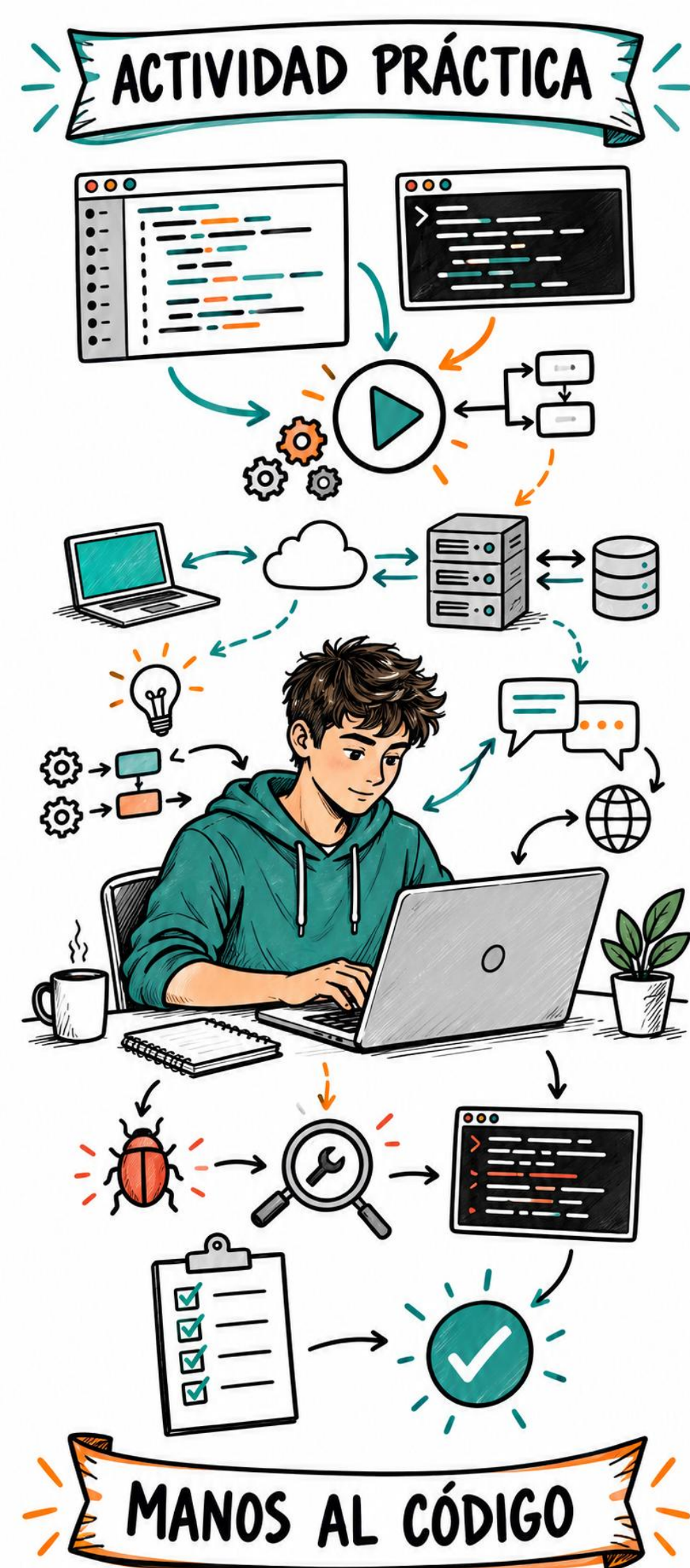
- Tareas que trabajan sobre los mismos datos → **HILOS**.
- Tareas independientes, o que deben aislarse por seguridad o por robustez → **PROCESOS**.

Chrome abre una pestaña por proceso precisamente para que una página rota no se lleve el navegador entero.

Hemos visto qué es un proceso y cómo lo gestiona el sistema. Ahora vamos a crearlos y controlarlos nosotros desde Java.

Actividad RA1_A02

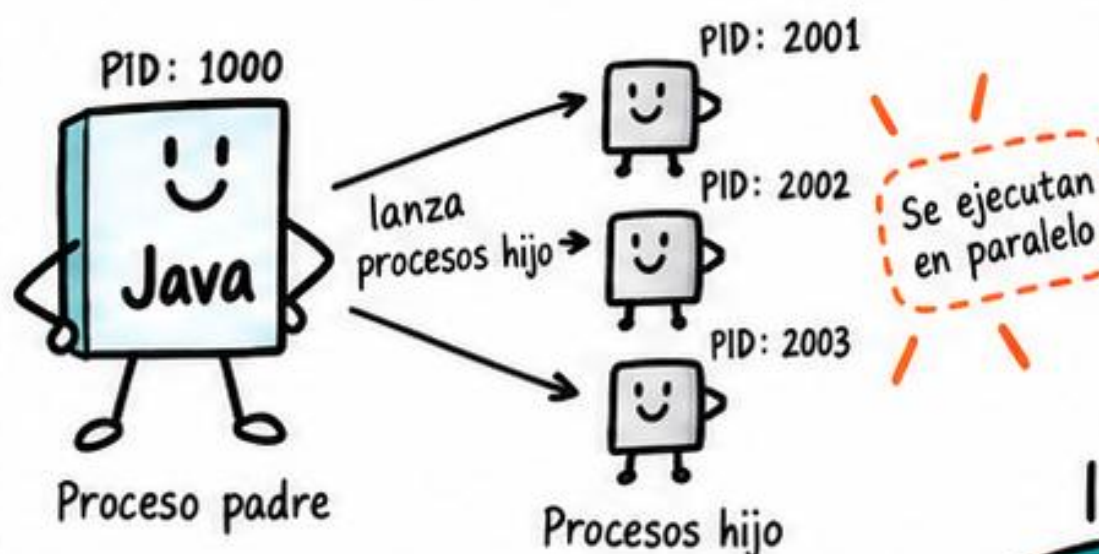
Radiografía de un proceso
en Windows





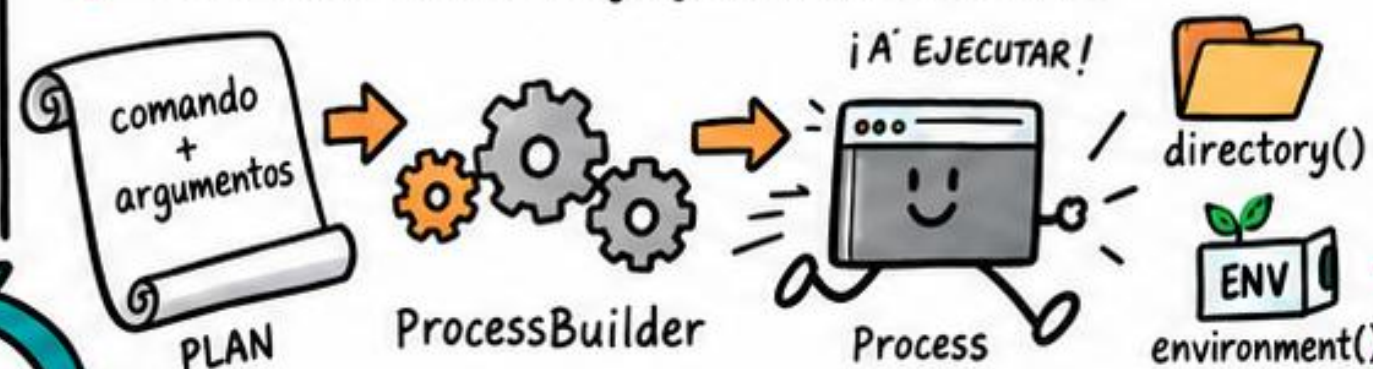
IDEA GENERAL

- Tu programa Java es el proceso padre
- Los programas que lanza son procesos hijo
- Padre e hijo se ejecutan en paralelo
- Cada proceso tiene su propio PID
- Al terminar, el hijo devuelve un código de salida



5.1 CREAR EL PROCESO

- ProcessBuilder: configura el comando
- Process: representa el proceso en marcha
- El comando va troceado: programa + argumentos
- directory(): fija la carpeta de trabajo
- environment(): ajusta variables de entorno
- start(): lanza el hijo y devuelve el control



¡OJO!

Runtime.exec(String)
está deprecado desde Java 18
Usa ProcessBuilder

5.2 COMUNICARSE CON EL HIJO

- Tres canales: stdin, stdout y stderr
- getOutputStream(): escribir en la entrada del hijo
- getInputStream(): leer su salida estándar
- getErrorStream(): leer su salida de error
- Puedes redirigir a consola o a archivos
- startPipeline(): conecta varios procesos



5.3 ESPERAR Y COMPROBAR

- waitFor(): bloquea hasta que termine
- waitFor(tiempo): evita esperas eternas
- Código de salida: 0 = correcto
- Código de salida: distinto de 0 = error
- onExit(): avisa cuando termina sin bloquear

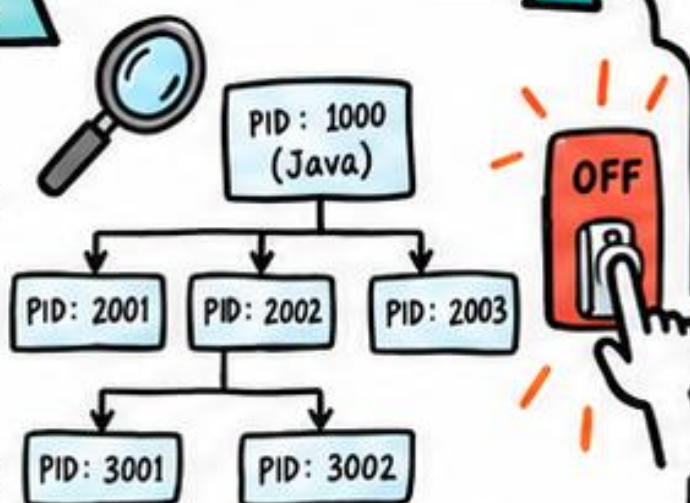


5. CREACIÓN Y GESTIÓN DE PROCESOS DESDE JAVA

De la configuración al control completo del proceso hijo

CONSULTAR Y TERMINAR

- ProcessHandle: visión del sistema
- info(): comando, usuario y hora de inicio
- children(): hijos directos
- descendants(): hijos, nietos y más
- destroy(): petición ordenada
- destroyForcibly(): terminación inmediata



BUENAS PRÁCTICAS

- Separa siempre programa y argumentos
- Cierra la entrada del hijo cuando acabes de escribir
- Lee la salida y los errores, o redirígelos
- Comprueba siempre el código final

¡PROCESOS BAJO CONTROL!

CONFIGURAR → LANZAR → COMUNICAR → ESPERAR → TERMINAR

5. Creación y gestión de procesos desde Java

5.1. Creación de procesos

5.1.1 Las clases Process y ProcessBuilder

Hasta ahora hemos gestionado procesos con las herramientas del sistema. Ahora los lanza nuestro código.

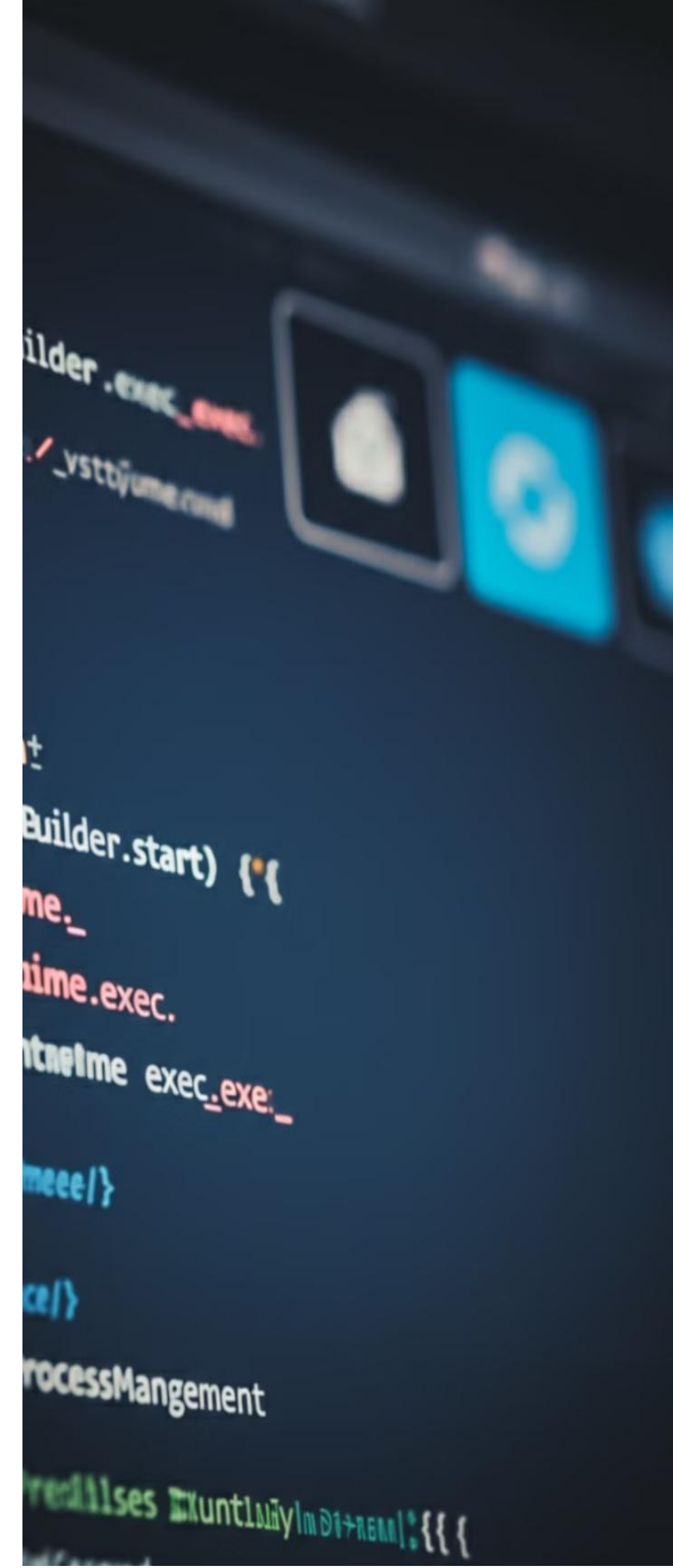
Dos clases con dos papeles principales:

- ☐ **ProcessBuilder** : configura y lanza. Es el plano.
- ☐ **Process** : representa el proceso ya en marcha. Es el proceso vivo.

Esquema mínimo:

```
ProcessBuilder pb = new ProcessBuilder("tasklist");  
Process p = pb.start();
```

 Verás por ahí `Runtime.getRuntime().exec("...")`. Está deprecado desde Java 18.
No lo uses



5.1.1 La clase ProcessBuilder

Se trata del objeto que guarda la configuración del proceso (**el plano del proceso**). Pero todavía no ha lanzado nada.

¿Qué se puede configurar antes de lanzar?

- El comando y sus argumentos.
- El directorio de trabajo.
- Las variables de entorno.
- A dónde van la entrada, la salida y los errores. (lo veremos en 5.2)

Se configura encadenando llamadas:

```
ProcessBuilder pb = new ProcessBuilder("tasklist", "/FO", "CSV");  
pb.directory(new File("C:\\practicass"));
```

Un objeto de la clase ProcessBuilder (**un plano**), puede lanzar varios procesos. El mismo ProcessBuilder puede llamar a start() varias veces y cada llamada crea un proceso nuevo e independiente..

 Igual que un ejecutable puede dar lugar a muchos procesos, un ProcessBuilder puede lanzar muchos hijos.



5.1.1 La clase Process

Process: el proceso ya en marcha.
Lo devuelve **start()**. Es tu mando a distancia sobre el proceso hijo.

¿Qué se puede consultar con Process?

- **pid()**: el identificador, el mismo que verías en el Administrador de tareas.
- **isAlive()**: si sigue en ejecución.
- **info()**: comando, usuario que lo lanzó y hora de inicio.

¿Qué nos permite hacer Process?

- Comunicarte con él a través de sus flujos → Apartado 5.2.
- Esperarlo y recoger lo que devuelve → Apartado 5.3



No se crea con **new**

Process es una clase abstracta. El objeto concreto lo fabrica **start()**, y depende del sistema operativo.



5.1.2 Argumentos, directorio de trabajo y entorno

Argumentos



Cada argumento va por separado, como elemento propio.

MAL

```
new ProcessBuilder("tasklist /FI \"IMAGENAME eq notepad.exe\");
```

Java busca un ejecutable que se llame literalmente así, pero NO EXISTE.

BIEN

```
new ProcessBuilder("tasklist", "/FI", "IMAGENAME eq notepad.exe");
```

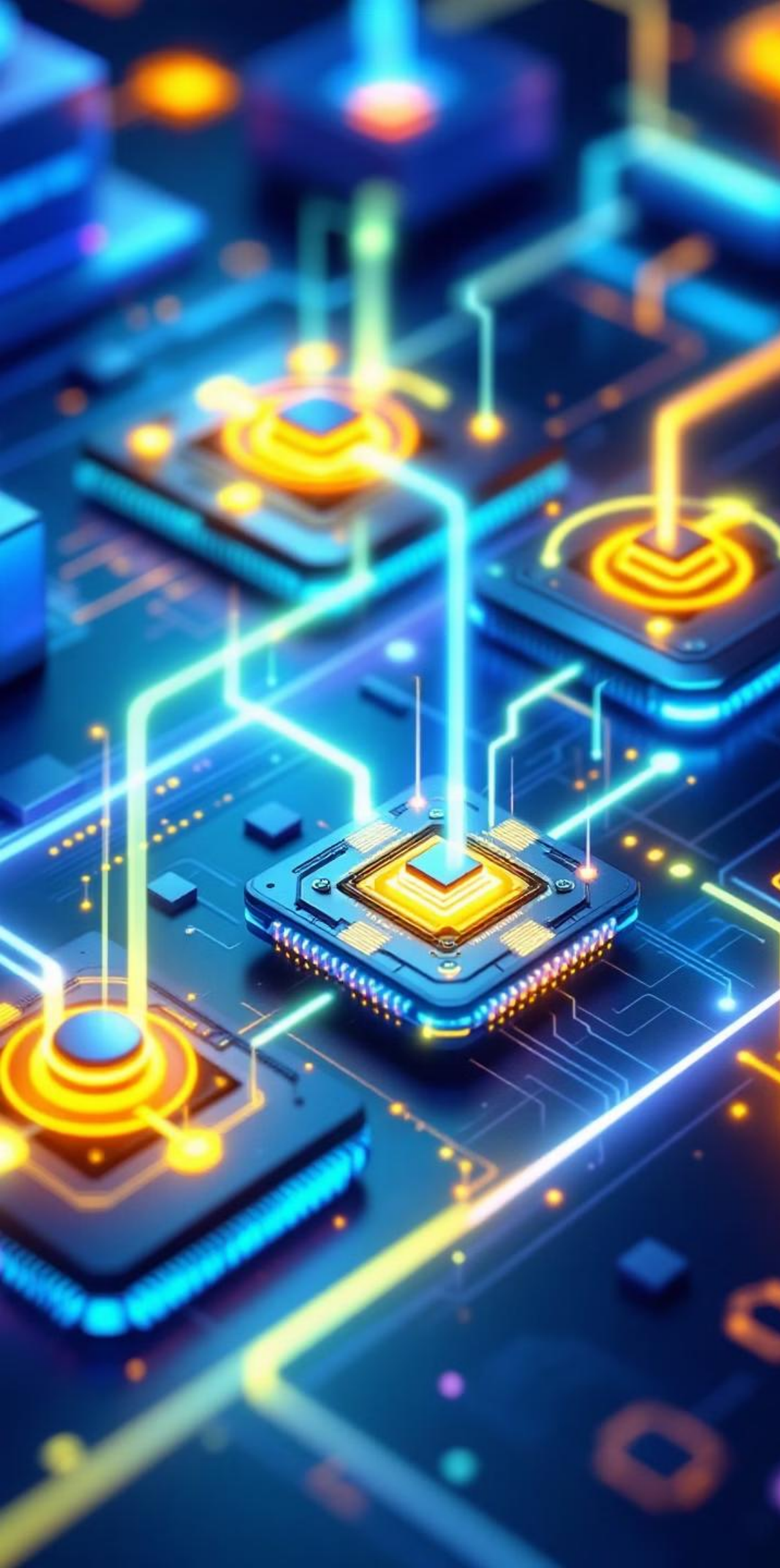
Tres elementos: el programa y sus dos argumentos.

VENTAJA

El tercer argumento lleva espacios dentro y no pasa nada: como no hay que trocear la cadena, no hacen falta comillas de escape ni hay ambigüedad posible.



Esta es exactamente la razón por la que se deprecó **Runtime.exec(String)**.



5.1.2 Argumentos, directorio de trabajo y entorno

Directorio

Desde qué carpeta se ejecuta el proceso hijo.

Por defecto, se hereda el directorio del proceso padre

Es decir, la carpeta desde la que se lanzó tu programa Java.
En IntelliJ, ese directorio coincide con la raíz del proyecto.

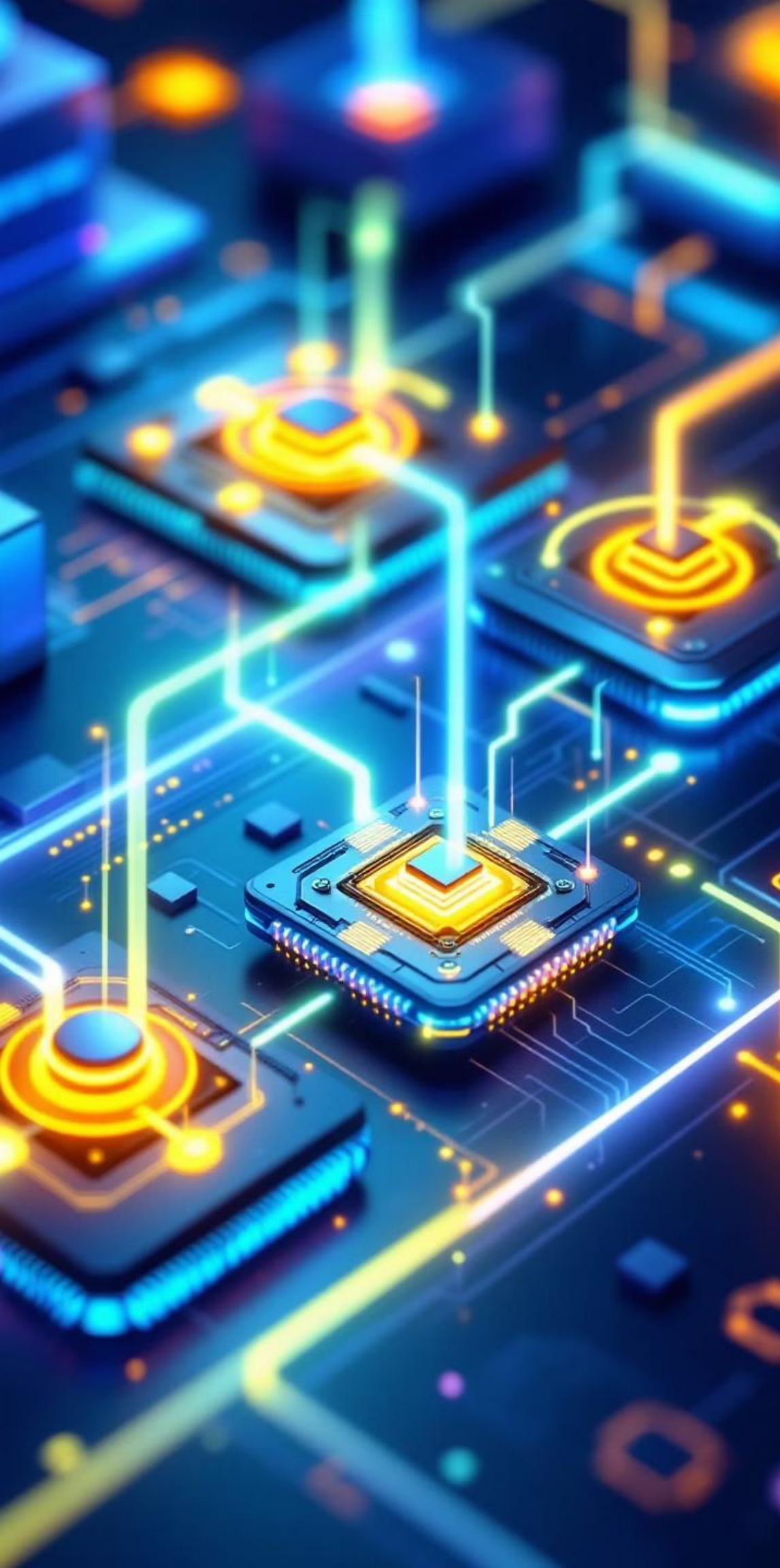
Se cambia con **directory()**

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "dir");  
pb.directory(new File("C:\\practicas"));  
Process p = pb.start();
```

Es importante tenerlo presente porque Todas las rutas relativas que use el hijo se resuelven desde ahí. Si el hijo escribe salida.txt, aparecerá en esa carpeta, no en la de tu proyecto.



Fallo típico: el programa "no genera el archivo". Sí lo genera, pero en otro sitio.



5.1.2 Argumentos, directorio de trabajo y entorno

Entorno

Datos de configuración que el sistema pasa a cada proceso.

PATH, TEMP, USERNAME... Son la forma estándar de configurar un programa sin tocar su código.

Se consultan y se modifican con **environment()**

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "echo %MOD0%");  
Map<String, String> entorno = pb.environment();  
entorno.put("MOD0", "PRUEBAS");  
Process p = pb.start();
```

Lo que cambias afecta solo al hijo:

environment() devuelve una copia. Ni el sistema ni tu propio programa Java se ven modificados.

Es la primera forma de pasarle información a un subprocesso.
En el apartado 5.2 veremos la otra: hablarle mientras se ejecuta.

5.1.3 Lanzamiento y ciclo de vida del proceso hijo

start(): el momento del lanzamiento

Hasta aquí solo había configuración. **start()** crea el proceso de verdad.

```
Process p = pb.start();  
System.out.println("PID del hijo: " + p.pid());  
System.out.println("¿Vivo? " + p.isAlive());
```

El padre no se detiene:

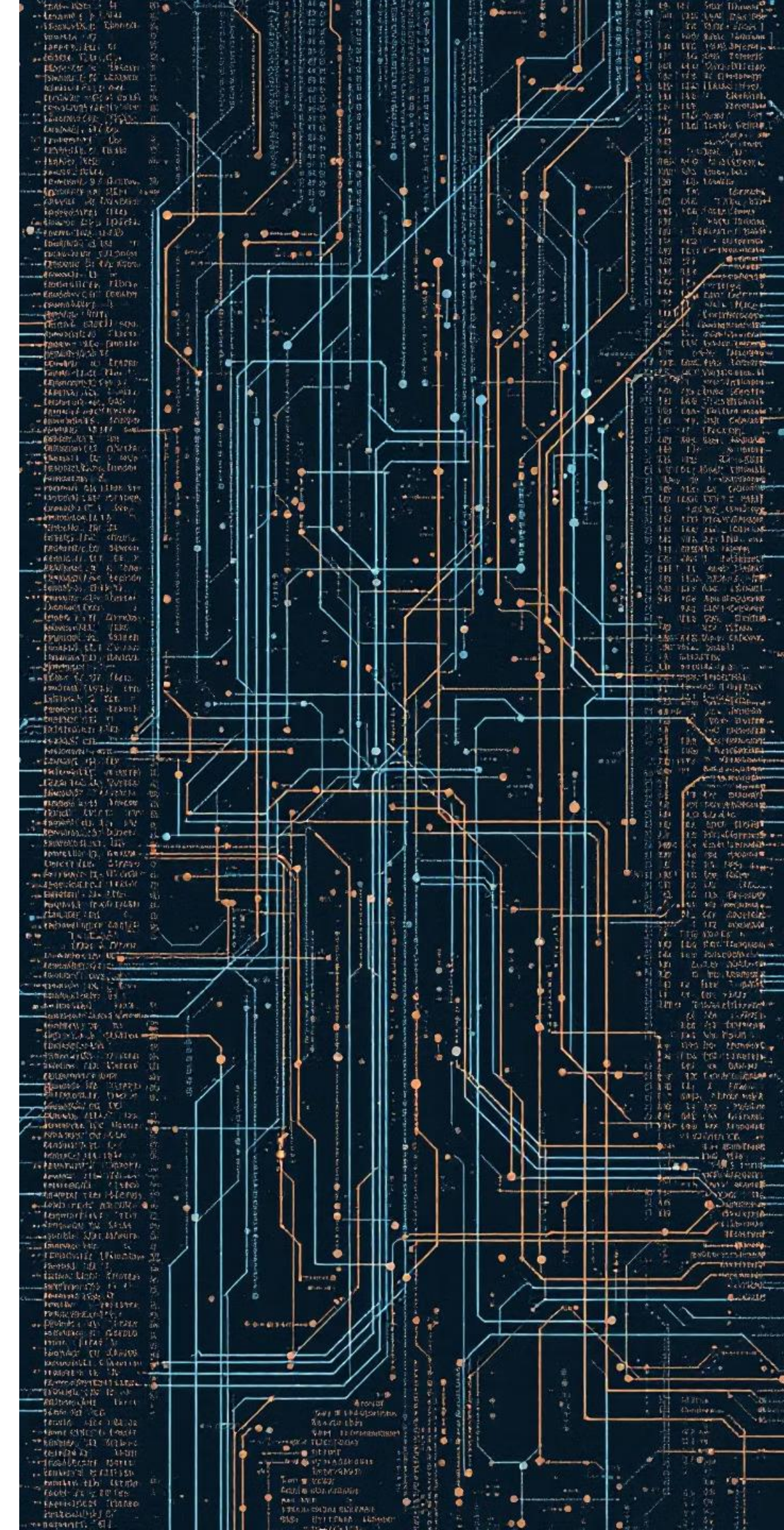
start() devuelve el control inmediatamente. A partir de ahí hay dos procesos ejecutándose a la vez: el tuyo y el hijo.

El ciclo de vida del hijo

Nace con **start()** → se ejecuta de forma independiente → termina por su cuenta → deja un código de salida.

El padre puede acabar antes que el hijo.

- Si quieres esperarlo y saber cómo le fue, necesitas el apartado 5.3.
- Y si quieres leer lo que ha escrito, el apartado 5.2.



Ejemplo: Proceso padre e hijos

```
import java.util.Scanner;
```

```
/**
```

```
 * Objetivo: ver quién es el proceso padre y quiénes son los hijos.
```

```
 * Ejecutar desde IntelliJ y NO cerrar la ventana: los hijos siguen vivos 20 segundos.
```

```
 */
```

```
public class PadreEHijos {
```

```
    private static final int NUM_HIJOS = 3;
```

```
    public static void main(String[] args) throws Exception {
```

```
        // ===== 1. EL PADRE =====
```

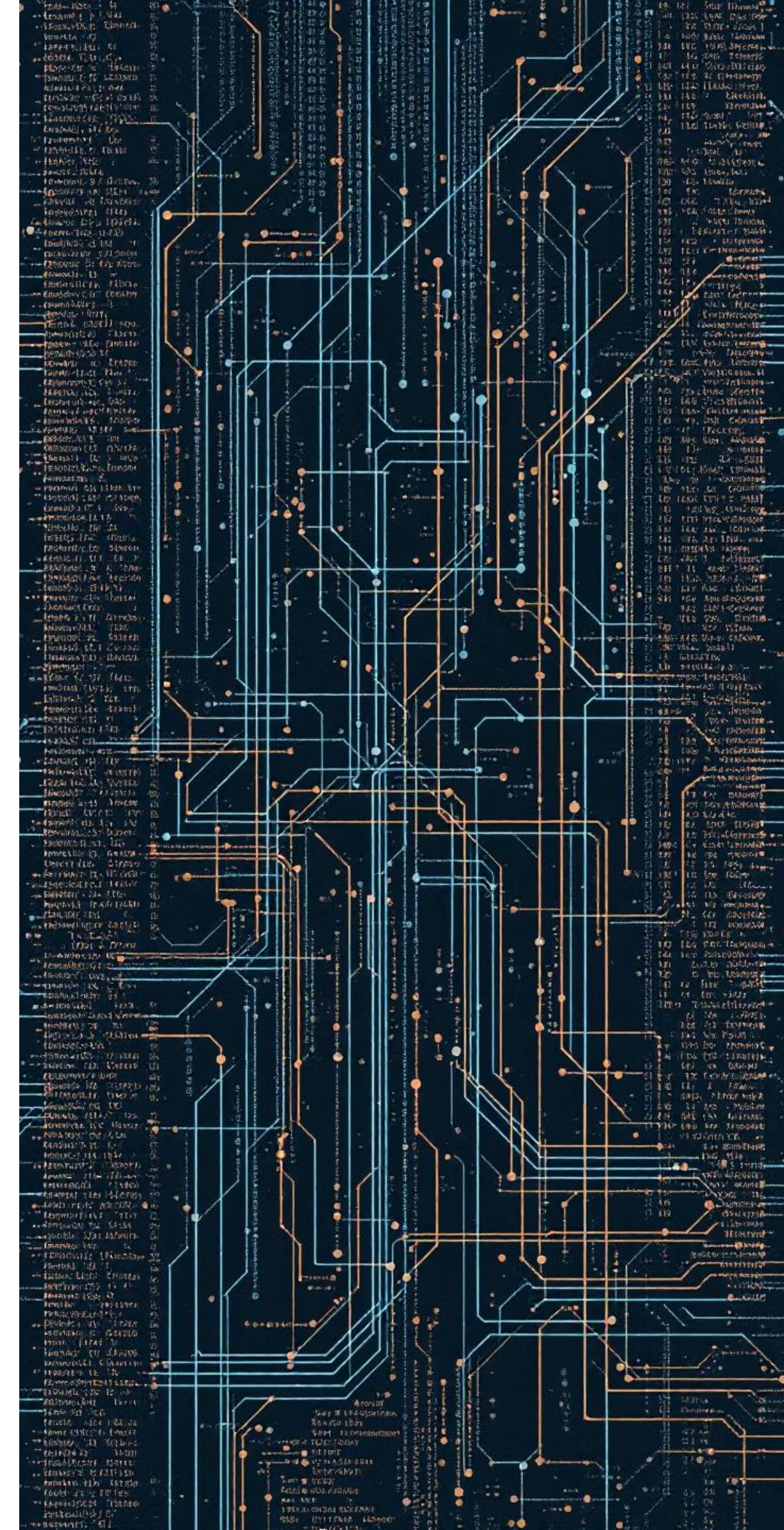
```
        // Este programa Java, al ejecutarse, ya ES un proceso: el proceso PADRE.
```

```
        // ProcessHandle.current() es el "mando a distancia" sobre nosotros mismos.
```

```
        long pidPadre = ProcessHandle.current().pid();
```

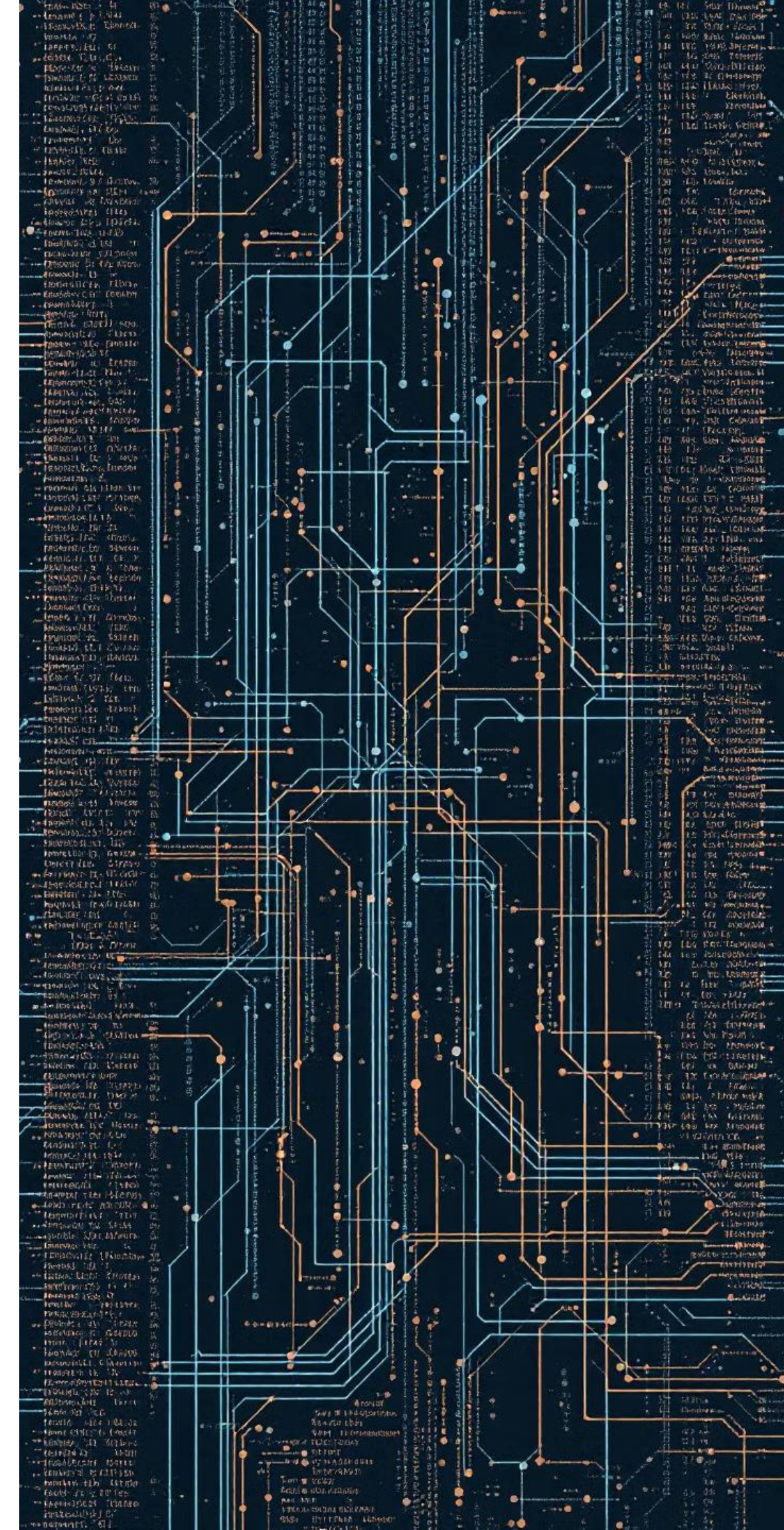
```
        System.out.println(" PADRE (este programa Java) -> PID " + pidPadre);
```

```
        long inicio = System.currentTimeMillis(); // Lo utilizamos para ver cuanto tiempo se ejecutan
```



Ejemplo: Proceso padre e hijos

```
// ===== 2. LOS HIJOS =====  
for (int i = 1; i <= NUM_HIJOS; i++) {  
  
    // El comando va troceado: cada argumento es un elemento de la lista.  
    // El hijo escribe un mensaje y despues se queda 20 segundos "ocupado"  
    // (el ping a 127.0.0.1 es solo una forma de hacerle esperar).  
    ProcessBuilder pb = new ProcessBuilder(  
        "cmd", "/c",  
        "echo --> Soy el HIJO " + i + " y estoy escribiendo en la consola de mi padre"  
        + " & ping -n 21 127.0.0.1 > nul");  
  
    // inheritIO(): el hijo usa la MISMA consola que el padre.  
    // Sin esta linea no veriamos nada de lo que escribe el hijo.  
    pb.inheritIO();  
  
    Process hijo = pb.start(); // <--- AQUÍ nace el proceso hijo  
  
    System.out.println(" HIJO " + i + " lanzado -> PID " + hijo.pid()  
        + " (vivo: " + hijo.isAlive() + ")");  
}  
  
long tardado = System.currentTimeMillis() - inicio;
```



Ejemplo: Proceso padre e hijos

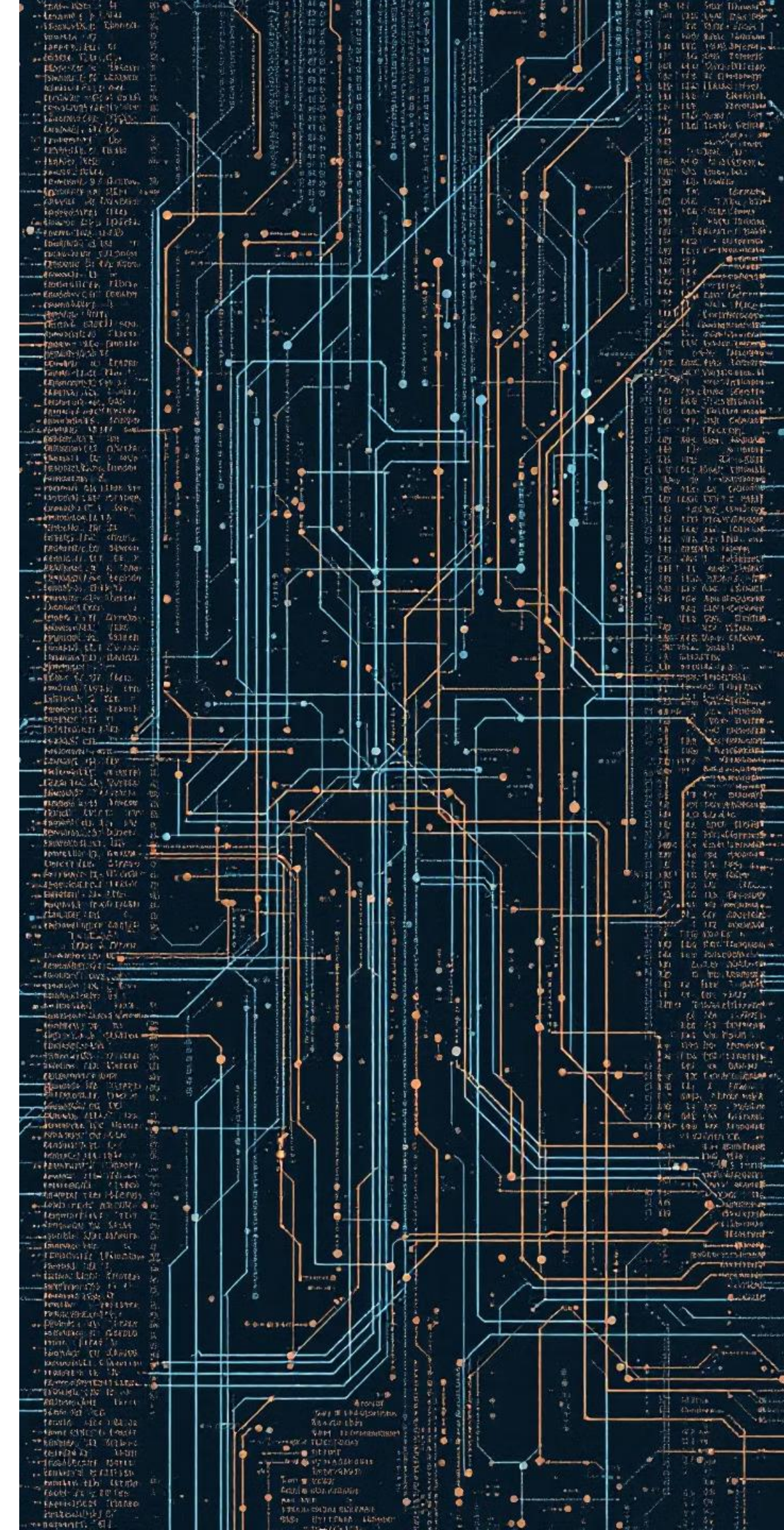
```
// ===== 3. LA FAMILIA =====
System.out.println("\n El padre ha tardado " + tardado + " ms en lanzar a los "
    + NUM_HIJOS + " hijos.");
System.out.println(" No los ha esperado: start() devuelve el control al instante.\n");

// Java sabe quienes son sus hijos directos:
System.out.println(" Hijos que el padre reconoce ahora mismo:");
ProcessHandle.current()
    .children()
    .forEach(h -> System.out.println(" - PID " + h.pid()));

// ===== 4. COMPROBACION =====
System.out.println("\n AHORA: abre el Administrador de tareas o escribe en una consola");
System.out.println(" tasklist /FI \"PID eq \" + pidPadre + "\"");
System.out.println(" y busca tambien los PID de los hijos.\n");
System.out.println(" Pulsa INTRO para que el padre termine...");

new Scanner(System.in).nextLine();

System.out.println("\n PADRE terminado. Ojo: los hijos pueden seguir vivos.");
}
}
```



5.2. Comunicación entre procesos

5.2.1. Entrada, salida y error estándar

Cada proceso tiene su propia memoria y no puede leer la del otro.

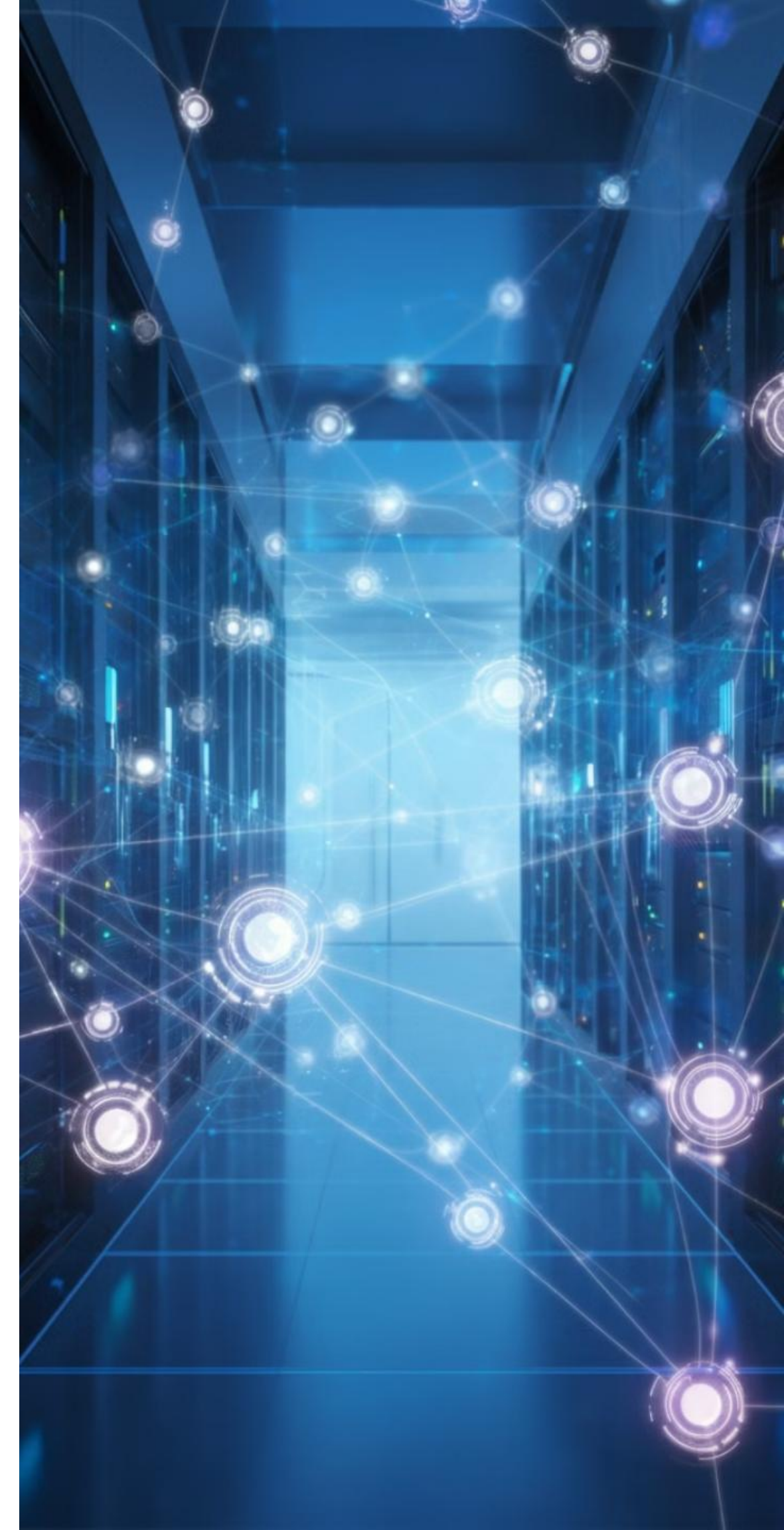
El sistema operativo lo impide por seguridad. Para intercambiar datos hacen falta canales.

Todo proceso nace con 3 canales abiertos

- **Entrada estándar (`stdin`):** de dónde lee lo que necesita. Normalmente el teclado, pero podría recibirlos de un fichero, de la tarjeta de red o hasta de otro proceso, entre otros sitios.
- **Salida estándar (`stdout`):** dónde escribe sus resultados. Normalmente la pantalla, aunque podría ser, la impresora o hasta otro proceso que necesite esos datos como entrada.
- **Salida de error (`stderr`):** dónde escribe sus mensajes de error. Normalmente también es la pantalla, pero se puede especificar otro canal aparte (por ejemplo un fichero de errores).

`stderr` puede ir separado porque permite quedarte con los resultados y descartar los avisos, o al revés. Si fueran el mismo canal, no podrías distinguirlos.

Estos tres canales son el mecanismo de comunicación entre procesos más antiguo y más usado que existe.



5.2. Comunicación entre procesos

5.2.2. Los flujos del hijo vistos desde el padre

El proceso hijo que lanza Java no tiene consola propia.
Sus tres canales quedan conectados al padre mediante tuberías. El padre es su único interlocutor.

Método de Process	Conecta con:	Lo usas para:
getOutputStream()	stdin del hijo	Escribirle
getInputStream()	stdout del hijo	Leer lo que produce
getErrorStream()	stderr del hijo	Leer sus errores

***Los nombres están vistos desde el padre, y eso puede confundir**



Regla para no equivocarse
los nombres describen tu flujo (proceso padre), no el del hijo.
Tu salida es su entrada.



5.2.2. Los flujos del hijo vistos desde el padre

Leer lo que escribe el hijo

El flujo de datos que lee el padre es de bytes, pero se envuelve para leerlo como texto, línea a línea.

```
Process p = new ProcessBuilder("tasklist").start();

try (BufferedReader br = new BufferedReader(new InputStreamReader(p.getInputStream()))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println("[hijo] " + linea);
    }
}
```

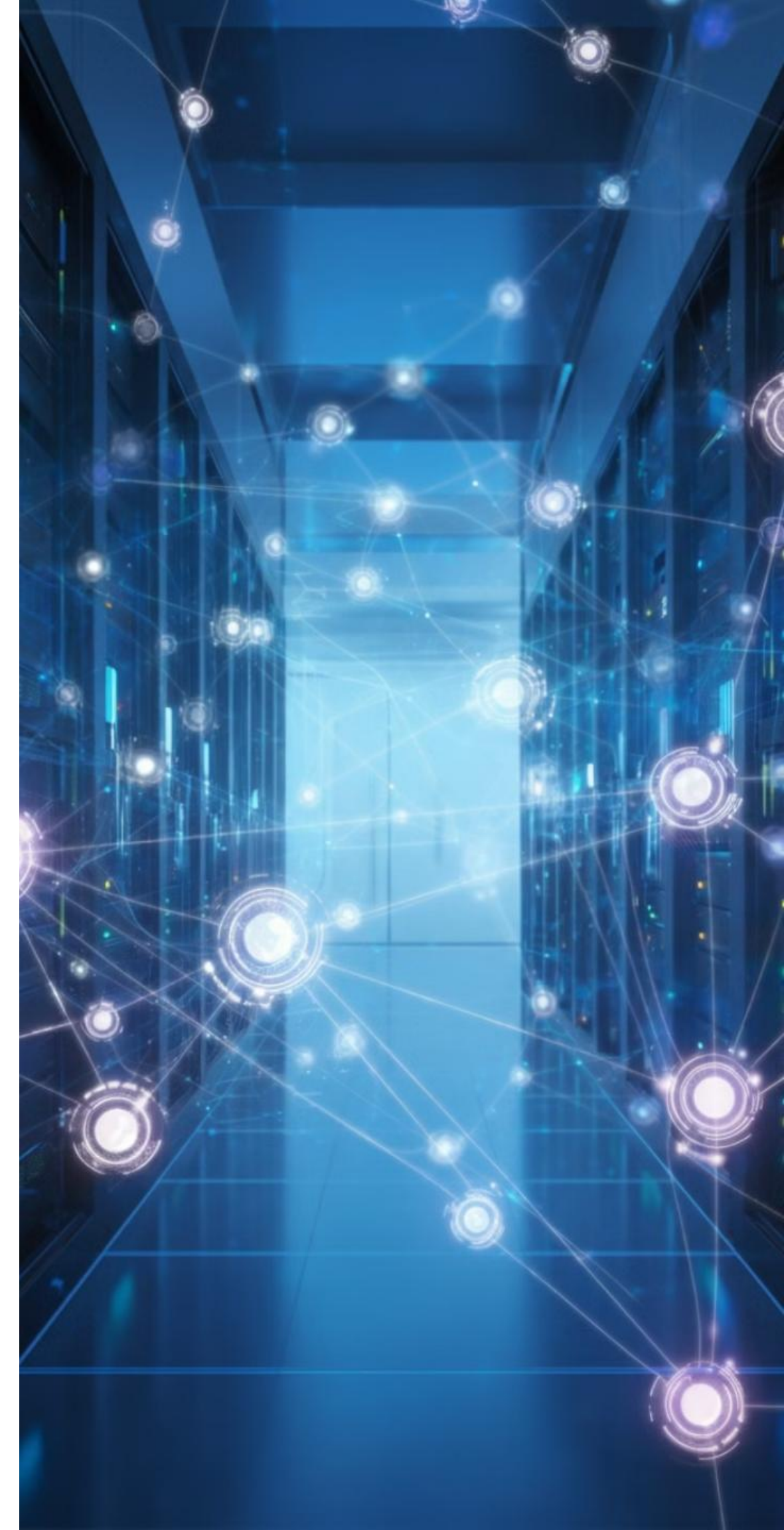
¿Qué hace cada capa?

- **getInputStream()**: da bytes.
- **InputStreamReader**: los convierte en caracteres.
- **BufferedReader**: permite leer líneas enteras.

readLine() devuelve null cuando el hijo cierra su salida, normalmente al terminar. Ese null es la señal de fin.



Si el hijo escribe mucho y nadie lee, la tubería se llena y el hijo se queda bloqueado para siempre. Leer no es opcional.



5.2.2. Los flujos del hijo vistos desde el padre

Leer los errores

El hijo puede fallar sin que te enteres.

Si solo lees `getInputStream()`, los mensajes de error se quedan en el otro canal, invisibles.

Opción A: juntar los dos canales antes de lanzar

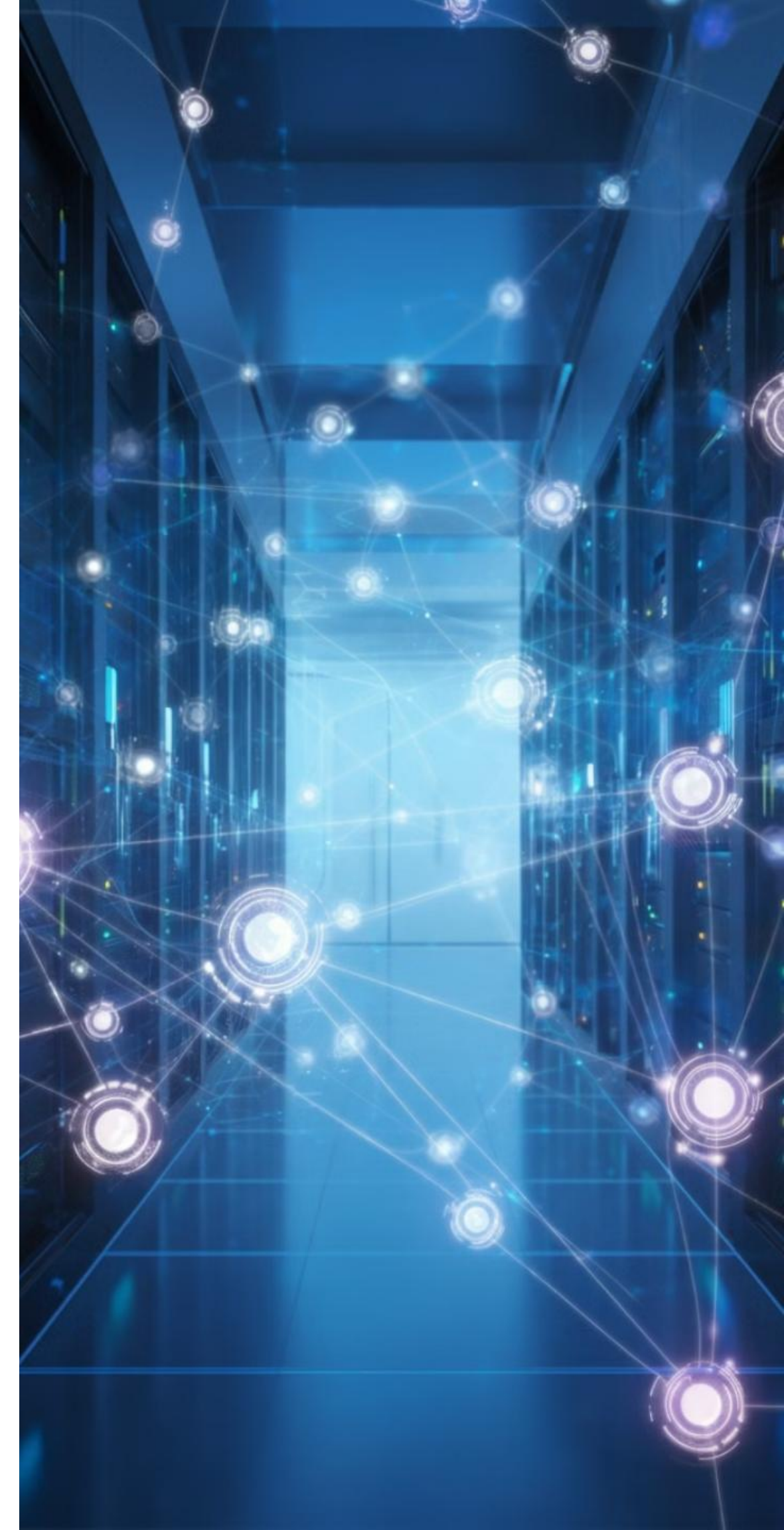
```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "dir zzz");  
pb.redirectErrorStream(true); // stderr pasa a stdout  
Process p = pb.start();      // ahora lo lees todo junto
```

Opción B: leerlos por separado

`getErrorStream()` con otro **`BufferedReader`**.

Los distingues, pero necesitas leer los dos a la vez o uno de ellos se llenará mientras esperas al otro.

Con **`redirectErrorStream(true)`** pierdes la distinción (si son errores o no), pero te ahorras el bloqueo. Para empezar, es la opción sensata.



5.2.2. Los flujos del hijo vistos desde el padre

Escribir en la entrada del hijo

Aquí es donde de verdad le pasamos información al subproceso.

sort es un comando de Windows que lee líneas de su entrada estándar y las devuelve ordenadas.

```
Process p = new ProcessBuilder("cmd", "/c", "sort").start();

// Escribimos EN LA ENTRADA del hijo
try (PrintWriter pw = new PrintWriter(p.getOutputStream())) {
    pw.println("pera");
    pw.println("manzana");
    pw.println("kiwi");
} // al cerrar, el hijo recibe el fin de entrada y se pone a trabajar
```

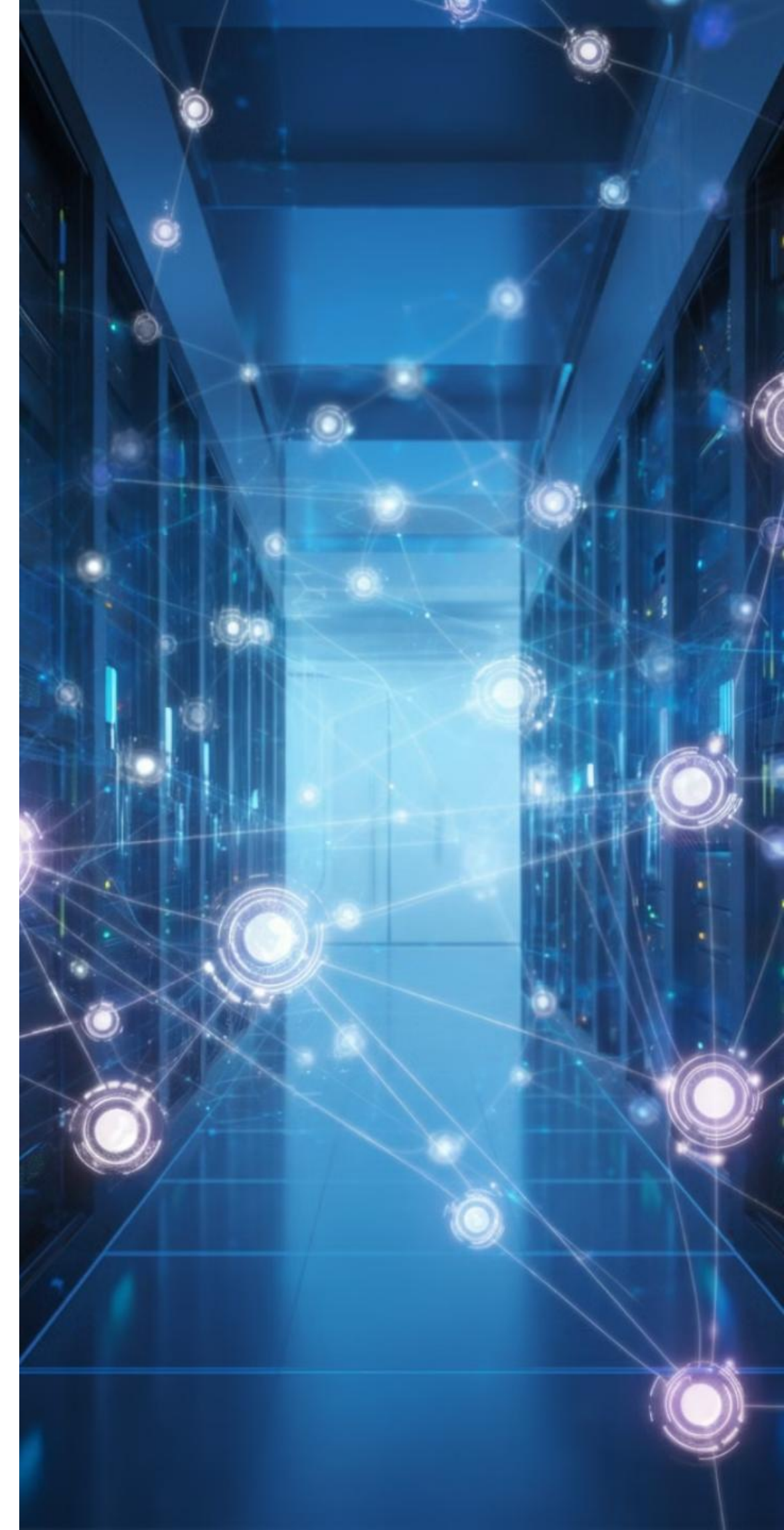
El cierre es imprescindible

Mientras el flujo siga abierto, el hijo cree que faltan datos por llegar y espera.
Cerrarlo es decirle "ya no hay más".



Escribir, cerrar, y después leer su salida.

Ese es el ciclo completo de comunicación con un subproceso.



5.2.3. Redirecciones y encadenamiento de procesos

Redirigir sin programar la lectura

A veces no quieres procesar la salida: solo que aparezca o que se guarde.

¿Qué podemos hacer?

Heredar la consola del padre

```
pb.inheritIO(); // el hijo escribe directamente en tu consola
```

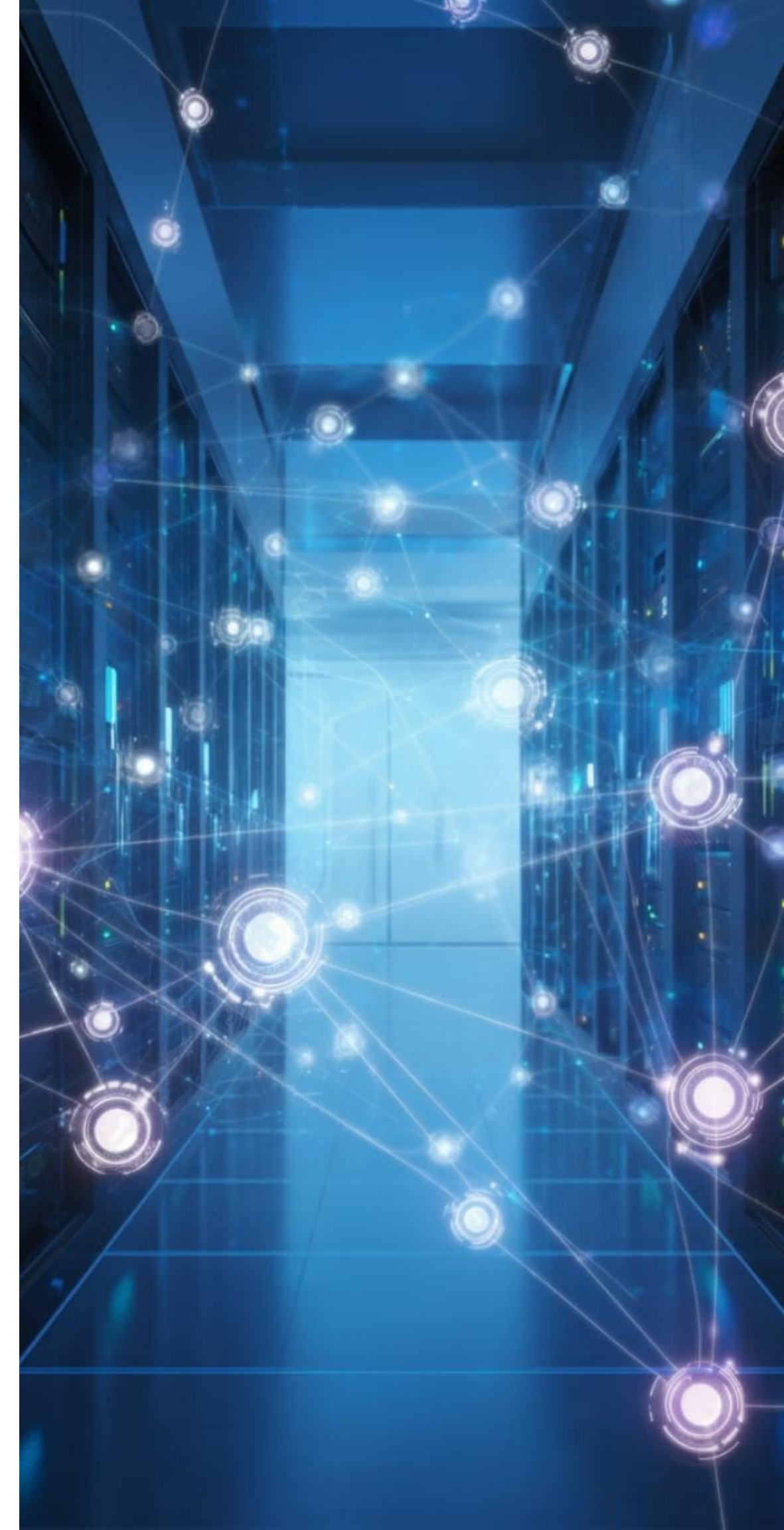
Redirigir a archivos

```
pb.redirectOutput(new File("salida.txt"));  
pb.redirectError(new File("errores.txt"));  
pb.redirectInput(new File("datos.txt"));
```

Añadir en lugar de sobrescribir

```
pb.redirectOutput(ProcessBuilder.Redirect.appendTo(new File("log.txt")));
```

Con redirección no hay tubería que se llene: el bloqueo desaparece.
Es lo más simple cuando no necesitas ver los datos desde Java.



5.2.3. Redirecciones y encadenamiento de procesos

Encadenar procesos

La salida de uno se conecta directamente con la entrada del siguiente.
Es la tubería de toda la vida de la consola, hecha desde Java.

```
List<Process> cadena = ProcessBuilder.startPipeline(List.of(  
    new ProcessBuilder("tasklist"),  
    new ProcessBuilder("findstr", "chrome")));  
  
Process ultimo = cadena.get(cadena.size() - 1);  
// de 'ultimo' se lee el resultado final, (leemos lo que escribe el hijo)
```

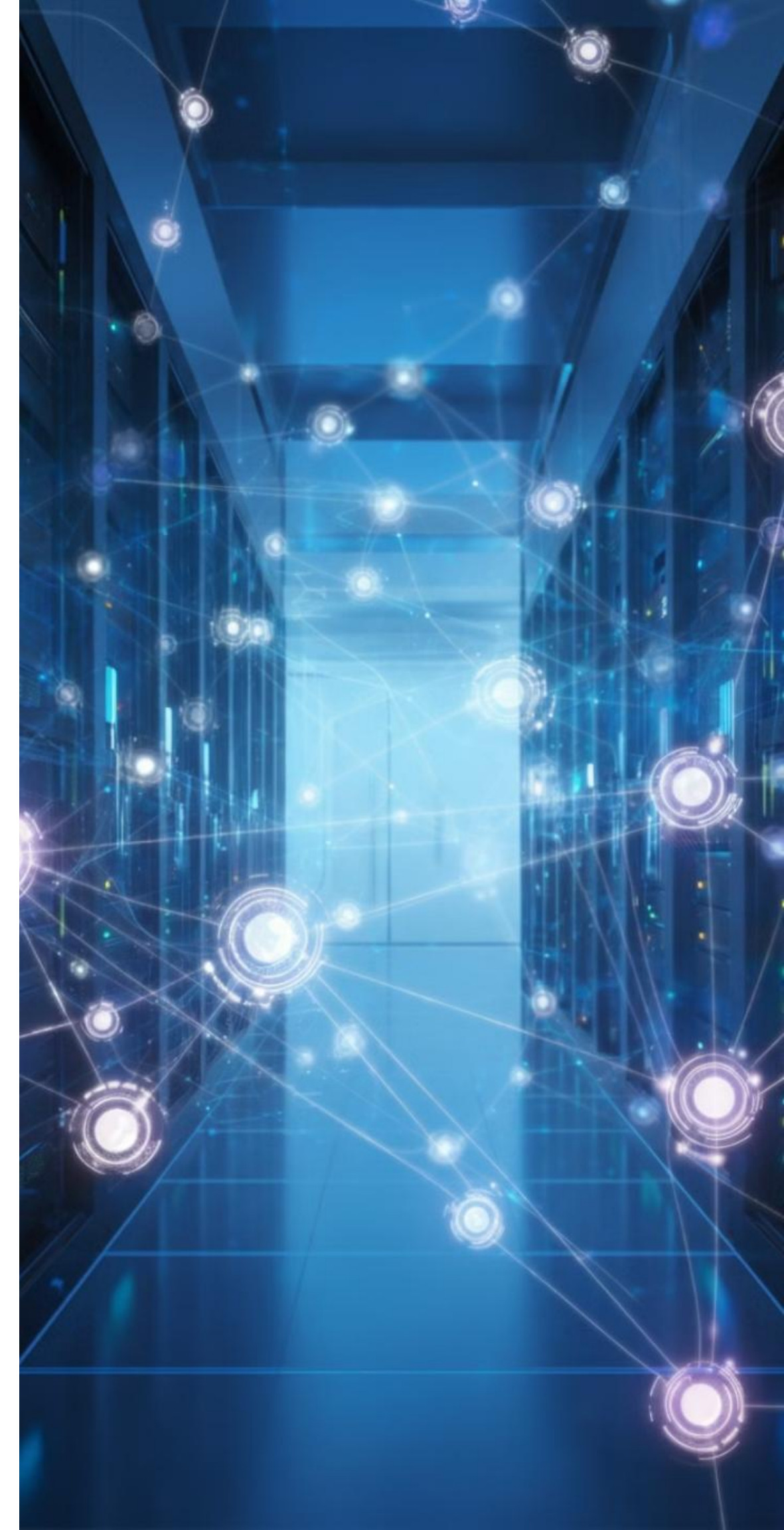
¿Qué hace **startPipeline()**?

Lanza todos los procesos a la vez y conecta la salida de cada uno con la entrada del siguiente.

Tú solo lees del último.

Disponible desde Java 9. Antes había que montar las tuberías a mano.

Ya sabemos hablar con el hijo. Falta lo más importante:
esperarlo y saber si ha terminado bien o mal. → apartado 5.3



Ejemplo: escritura al hijo y lectura de lo que devuelve

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
/**
 * Hablamos con un proceso hijo ('sort' de Windows) y escuchamos su respuesta.
 */
public class HablarConElHijo {

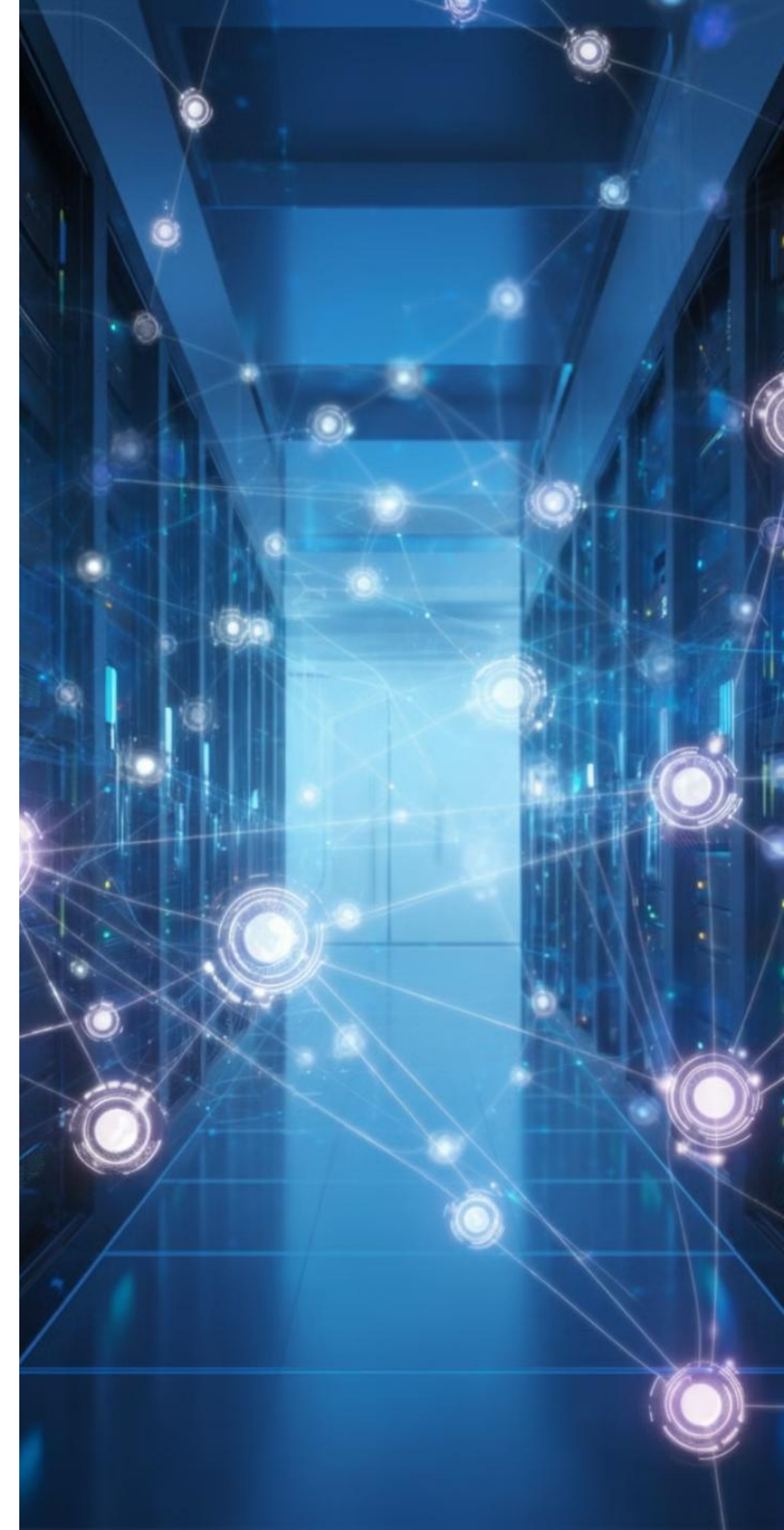
    public static void main(String[] args) throws Exception {

        Process hijo = new ProcessBuilder("sort").start();

        // getOutputStream es NUESTRA salida... y la ENTRADA del hijo.
        // El cierre automatico del try le avisa de que no hay mas datos.
        try (PrintWriter haciaElHijo = new PrintWriter(hijo.getOutputStream())) {
            for (String palabra : new String[]{"pera", "manzana", "kiwi", "cereza"}) {
                System.out.println(" -> " + palabra);
                haciaElHijo.println(palabra);
            }
        }

        // getInputStream es NUESTRA entrada... y la SALIDA del hijo.
        BufferedReader desdeElHijo = new BufferedReader(
            new InputStreamReader(hijo.getInputStream()));

        String linea;
        while ((linea = desdeElHijo.readLine()) != null) {
            System.out.println(" <- " + linea);
        }
    }
}
```



5.3. Sincronización entre procesos

El proceso padre no espera a nadie

start() devuelve el control al instante.

A partir de ahí hay dos procesos corriendo en paralelo, cada uno a su ritmo.

El problema

```
public class ProblemaSincronizacion {  
    public static void main(String[] args) throws Exception {  
        Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();  
        System.out.println("Ya he terminado"); // se imprime de inmediato  
    }  
}
```

***El mensaje aparece cuando el hijo apenas ha empezado.**

¿Cuándo esto es realmente un problema?:

- Si el hijo genera un archivo que necesitas leer después.
- Si el resultado de un proceso alimenta al siguiente.
- Si quieres saber si el trabajo salió bien.

Sincronizar aquí significa esperar y recoger el resultado. En la UT2 la palabra significará otra cosa.



5.3.1 Espera del proceso hijo: **waitFor()**

■ **waitFor():** esperar al hijo

Detiene al padre hasta que el hijo termina.

Solución: esperar

```
public class EsperaAHijo {  
    public static void main(String[] args) throws Exception {  
        Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();  
        int codigo = p.waitFor();    // el padre se queda aqui parado  
        System.out.println("El hijo ha terminado con codigo " + codigo);  
    }  
}
```

¿Qué le pasa al padre mientras espera?:

- Pasa al estado bloqueado que estudiamos en 2.1.
- No consume procesador: espera un evento externo, igual que si esperara al disco.

Esperar con límite de tiempo

```
boolean aTiempo = p.waitFor(10, TimeUnit.SECONDS);
```

Devuelve **false** si se agotó el plazo y el hijo sigue vivo. Evita que un hijo colgado cuelgue también al padre.

waitFor() lanza **InterruptedException**. Hay que declararla o capturarla.



5.3.2 Códigos de salida

Todo proceso devuelve un número entero al terminar.
Es su parte del resultado.

El convenio es universal:

- **0** : todo ha ido bien.
- **Distinto de 0** : algo ha fallado. Cada programa define el significado de cada número.

Dos formas de obtener el código:

- **waitFor()** : espera y devuelve el código.
- **exitValue()** : lo devuelve sin esperar, pero lanza excepción si el hijo aún vive.

Tu programa Java también devuelve su código:

System.exit(3);

termina la JVM con código 3. Así un Java puede informar a otro.

```
if (p.waitFor() == 0) {  
    System.out.println("Correcto");  
} else {  
    System.out.println("El hijo ha fallado");  
}
```

Puedes comprobar el último código de salida en la consola de Windows con
echo %ERRORLEVEL%



5.3.3 Espera asíncrona: **onExit()**

Esperar sin bloquearse.

A veces no quieres parar el padre, solo que te avise cuando el hijo acabe.

Ejemplo

```
Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();
p.onExit().thenAccept(proceso ->
    System.out.println("Aviso: el hijo " + proceso.pid() + " ha terminado"));
System.out.println("El padre sigue trabajando mientras tanto...");
```

¿Qué devuelve **onExit()**?

- Un **CompletableFuture<Process>**: una promesa de que ese proceso terminará.
- Con **thenAccept()** indicas qué hacer cuando ocurra.

La diferencia con **waitFor()**:

- **waitFor()** bloquea al padre.
- **onExit()** no bloquea al padre: registra el aviso y sigue.

Si el padre termina antes que el hijo, el aviso no llega a ejecutarse.
Para probarlo, añade un **waitFor()** al final.





5.3.4. Consulta y terminación: ProcessHandle y destroy()

ProcessHandle: la vista del sistema

Process controla un proceso que tú creaste. **ProcessHandle** ve todos los del sistema.

El proceso actual, tú mismo

```
ProcessHandle yo = ProcessHandle.current();  
System.out.println("Mi PID: " + yo.pid());
```

Buscar cualquier proceso por su PID

```
Optional<ProcessHandle> ph = ProcessHandle.of(4820);
```

* Devuelve Optional porque ese PID puede no existir.

Información disponible

- **info().command()** : ruta del ejecutable.
- **info().startInstant()** : cuándo arrancó.
- **info().user()** : quién lo lanzó.

Es la versión programable de lo que hacías a mano con tasklist en el apartado 3.



5.3.4. Consulta y terminación: ProcessHandle y destroy() Hijos, nietos y descendientes

El sistema organiza los procesos en un árbol de padres e hijos.

```
ProcessHandle.current().children().forEach(h -> System.out.println("Hijo: " + h.pid()));  
ProcessHandle.current().descendants().forEach(h -> System.out.println("Descendiente: " + h.pid()));
```

La diferencia:

- **children()** : solo los hijos directos.
- **descendants()** : hijos, nietos y toda la rama por debajo.

Un caso real:

Si lanzas **cmd /c ping**, tu hijo es cmd y ping es tu nieto.
En **children()** no aparece; en **descendants()** sí.

children() devuelve lo que el sistema considera tus hijos, no solo los que tú creaste con **start()**.



5.3.4. Consulta y terminación: ProcessHandle y destroy()

Terminar un proceso desde Java

Existen dos formas, exactamente igual que con taskkill

```
p.destroy();      // petición ordenada: puede cerrar y guardar  
p.destroyForcibly(); // terminación inmediata: sin margen
```

Es necesario comprobar que ha muerto de verdad: **destroy()** es una petición, no una garantía.

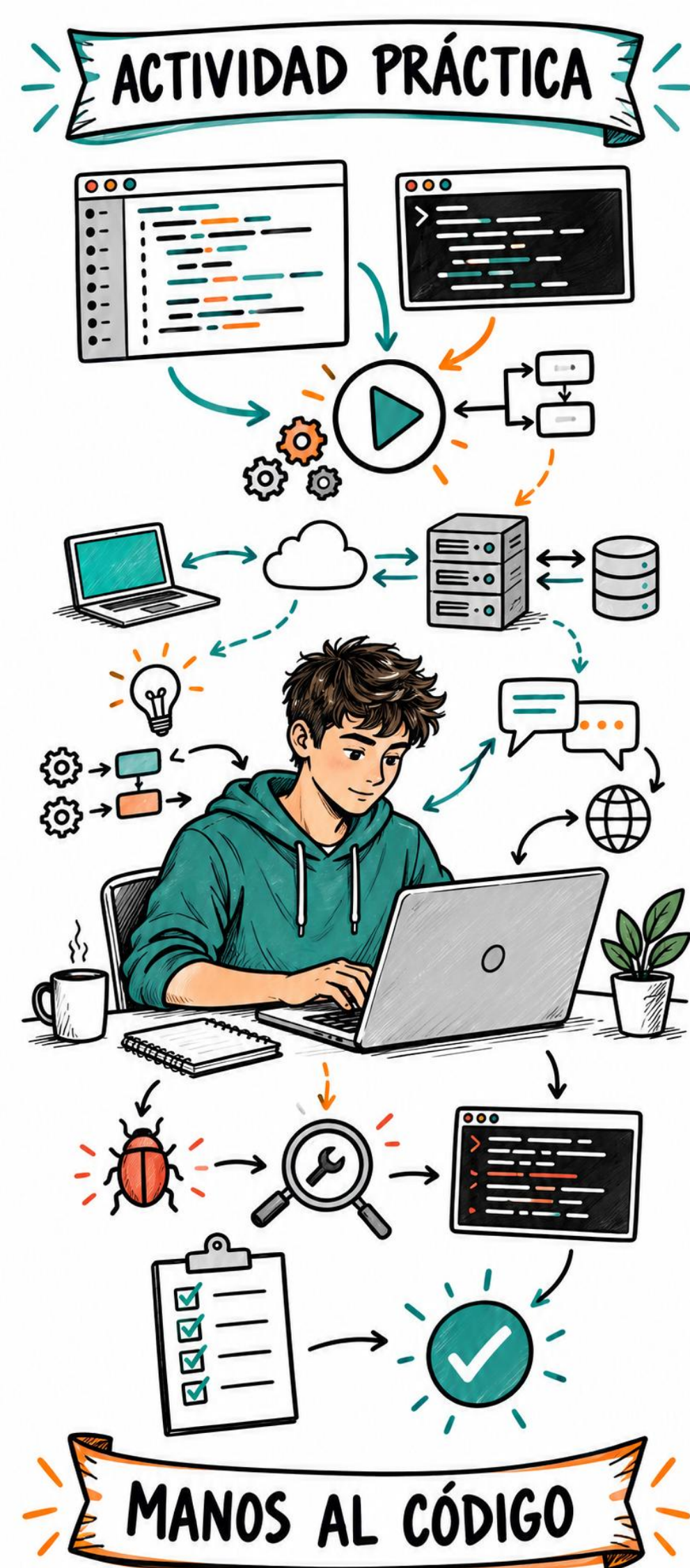
```
p.destroy();  
if (!p.waitFor(5, TimeUnit.SECONDS)) {  
    p.destroyForcibly();  
}
```

Es el equivalente programado de **taskkill /PID** y **taskkill /F** del apartado 3.

Ya sabemos crear, comunicar, esperar y terminar procesos: vamos a repartir trabajo de verdad entre ellos. → Apartado 6

Actividad RA1_A03

Lanzar subprocesos desde
Java y comunicarse con ellos



6. Aplicaciones multiproceso en paralelo

6.1. Reparto de una tarea entre varios subprocesos

Idea principal: dividir para ir más rápido

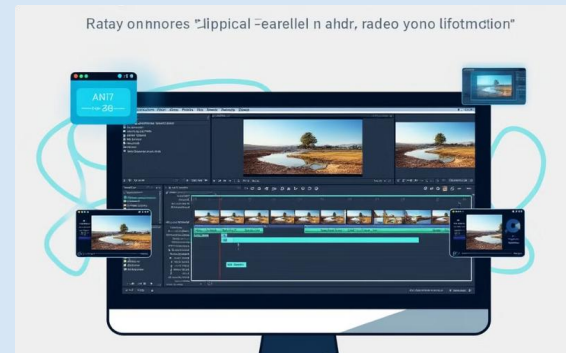
Dividir una tarea grande en partes independientes que se ejecutan a la vez.

Un mismo patrón de funcionamiento en 3 partes:

- **Dividir:** partir el trabajo en trozos que no dependan unos de otros.
- **Lanzar:** un subproceso por trozo, todos a la vez.
- **Recoger:** esperar a todos y juntar los resultados.

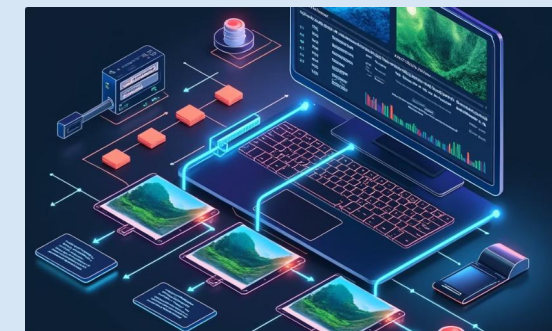
Casos reales

Procesamiento de imágenes



Filtros y mejoras de imagen en paralelo aceleran el proceso.

Simulaciones científicas



Modelos matemáticos y simulaciones físicas

La clave está en el primer paso. Si los trozos dependen entre sí, no hay paralelismo que valga.

6.1. Reparto de una tarea entre varios subprocessos

Idea principal: lanzar bucles separados

El error que lo estropea todo.
Lanzar primero, esperar después

```
for (int i = 0; i < 4; i++) {  
    Process p = pb.start();  
    p.waitFor();    // MAL: espera a cada uno antes de lanzar el siguiente  
}
```

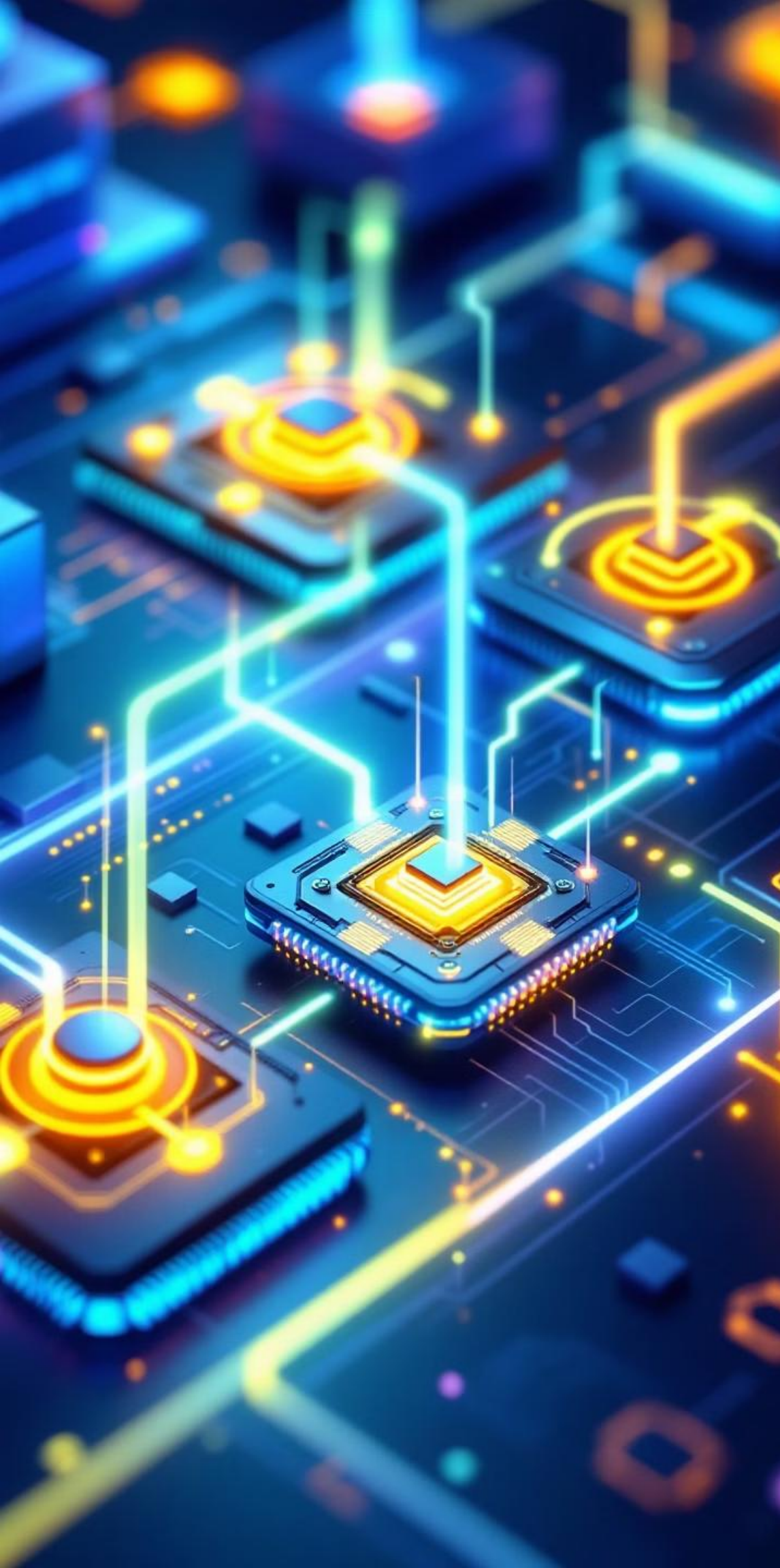
Aquí no hay paralelismo: los cuatro se ejecutan uno detrás de otro.

La forma correcta:
dos bucles separados

```
for (int i = 0; i < 4; i++) {    // 1) lanzar todos  
    procesos.add(new ProcessBuilder(...).start());  
}  
for (Process p : procesos) {    // 2) esperarlos después  
    p.waitFor();  
}
```



Lanzar y esperar tienen que ir en bucles distintos.
Es el fallo más frecuente de esta práctica.



6.1. Reparto de una tarea entre varios subprocesos

Idea principal: recoger los resultados comprobándolos

- No basta con esperar: hay que saber si cada parte salió bien.

```
int fallos = 0;
for (Process p : procesos) {
    if (p.waitFor() != 0) fallos++;
}
System.out.println("Subprocesos fallidos: " + fallos);
```

¿Dónde dejan sus resultados los subprocesos?

Cada subproceso escribe en su propio archivo de salida, redirigido con `redirectOutput()`.
Después el padre los junta.

¿Por qué no comparten un solo archivo?

Cuatro procesos escribiendo a la vez en el mismo archivo producen líneas entremezcladas o pérdidas.
Se crea un archivo por subproceso y una unión al final.

**Cada proceso tiene su memoria: no hay variables compartidas.
Comunicarse cuesta, y por eso se reparte trabajo grande, no tareas diminutas.**

6.2. Medición frente a la ejecución secuencial

Necesitamos medir para ver si los procesos en paralelo hacen el trabajo mas rápido

```
long inicio = System.nanoTime();  
// ... lanzar y esperar a todos ...  
long ms = (System.nanoTime() - inicio) / 1_000_000;  
System.out.println("Tiempo: " + ms + " ms");
```

¿Cómo hacer la comparación bien?

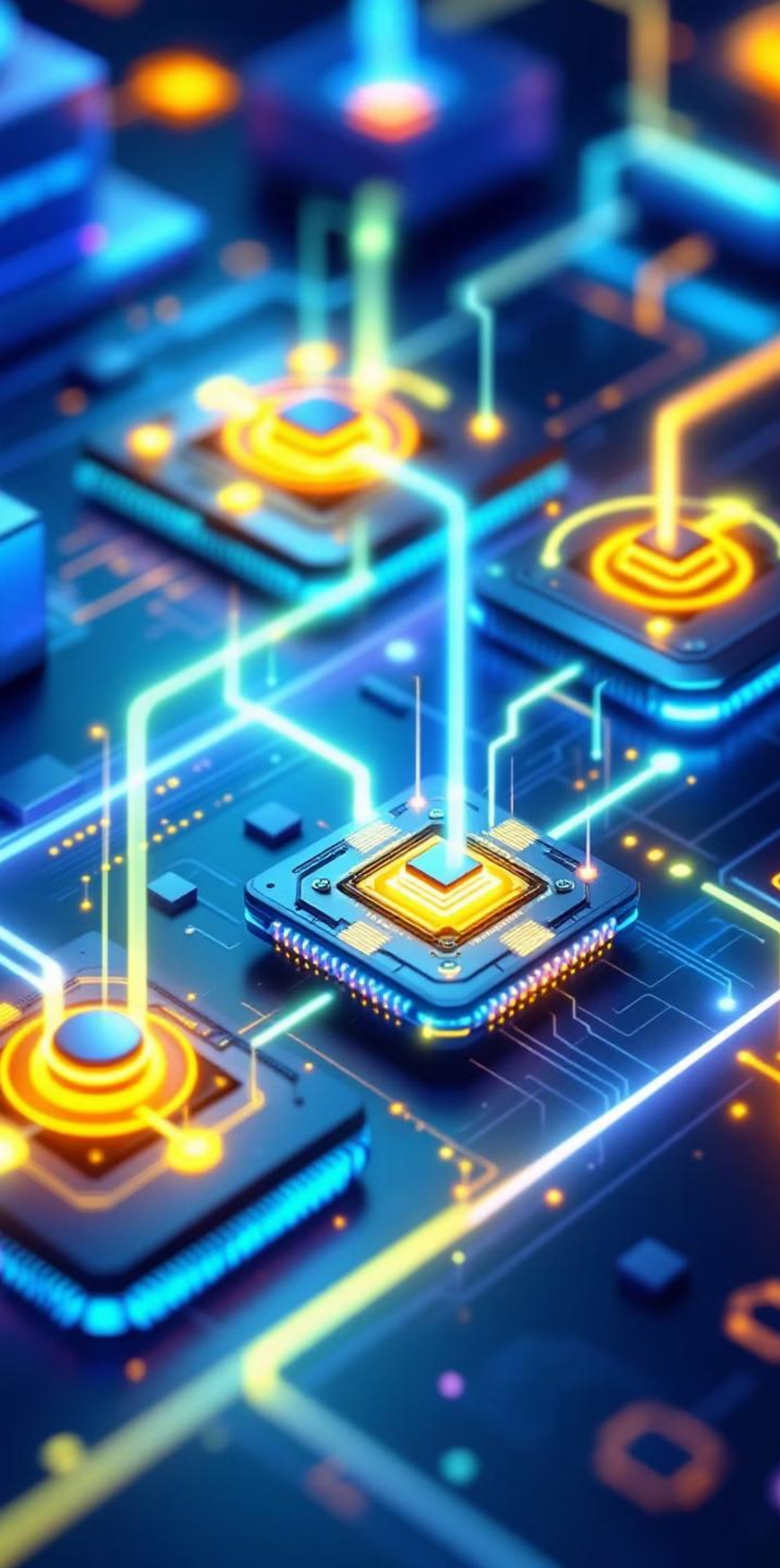
1. Mismo trabajo, misma máquina, mismos datos.
2. Primero la versión secuencial, después la paralela.
3. Repetir tres veces cada una y quedarse con la mediana.

La fórmula

Aceleración = tiempo secuencial ÷ tiempo paralelo.

Si sale 2,5, la versión paralela es dos veces y media más rápida.

nanoTime() es para medir intervalos. **currentTimeMillis()** es para saber la hora.
No los confundas.



6.2. Medición frente a la ejecución secuencial

¿Por qué nunca sale una velocidad proporcional al número de núcleos?

- **Crear cada proceso cuesta tiempo, y son varios procesos.**
- **Siempre queda una parte que no se puede repartir:** preparar los datos y juntar los resultados.
- **Disco y memoria son compartidos:** cuatro procesos leyendo a la vez se estorban.

¿Cuándo merece la pena?

Solo si cada trozo dura bastante más de lo que cuesta crear el proceso. Para tareas de milisegundos, el paralelismo sale más caro que el trabajo.

¿Y si pides más procesos que núcleos?

No mejora: el planificador vuelve a repartir por turnos, como en el apartado 2.

Primero mide, después decide. Paralelizar sin medir suele empeorar las cosas.

Continuidad: ya tenemos aplicaciones que reparten trabajo entre varios procesos. Falta dejarlas entendibles y comprobables. → **Apartado 7**

Ejemplo

```
import java.util.ArrayList;
import java.util.List;

/**
 * La misma carga de trabajo, hecha de dos formas: secuencial o paralela.
 * Cada "tarea" son 5 segundos de espera (un ping que no imprime nada).
 */
public class SecuencialVsParalelo {

    private static final int N = 4;

    // Una tarea = 5 segundos ocupado
    private static ProcessBuilder tarea() {
        return new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1 > nul");
    }

    public static void main(String[] args) throws Exception {
```

Ejemplo

```
// ----- SECUENCIAL: lanzar y esperar, uno a uno -----
long t0 = System.nanoTime();
for (int i = 0; i < N; i++) {
    tarea().start().waitFor();    // espera antes de lanzar el siguiente
}
long secuencial = (System.nanoTime() - t0) / 1_000_000;

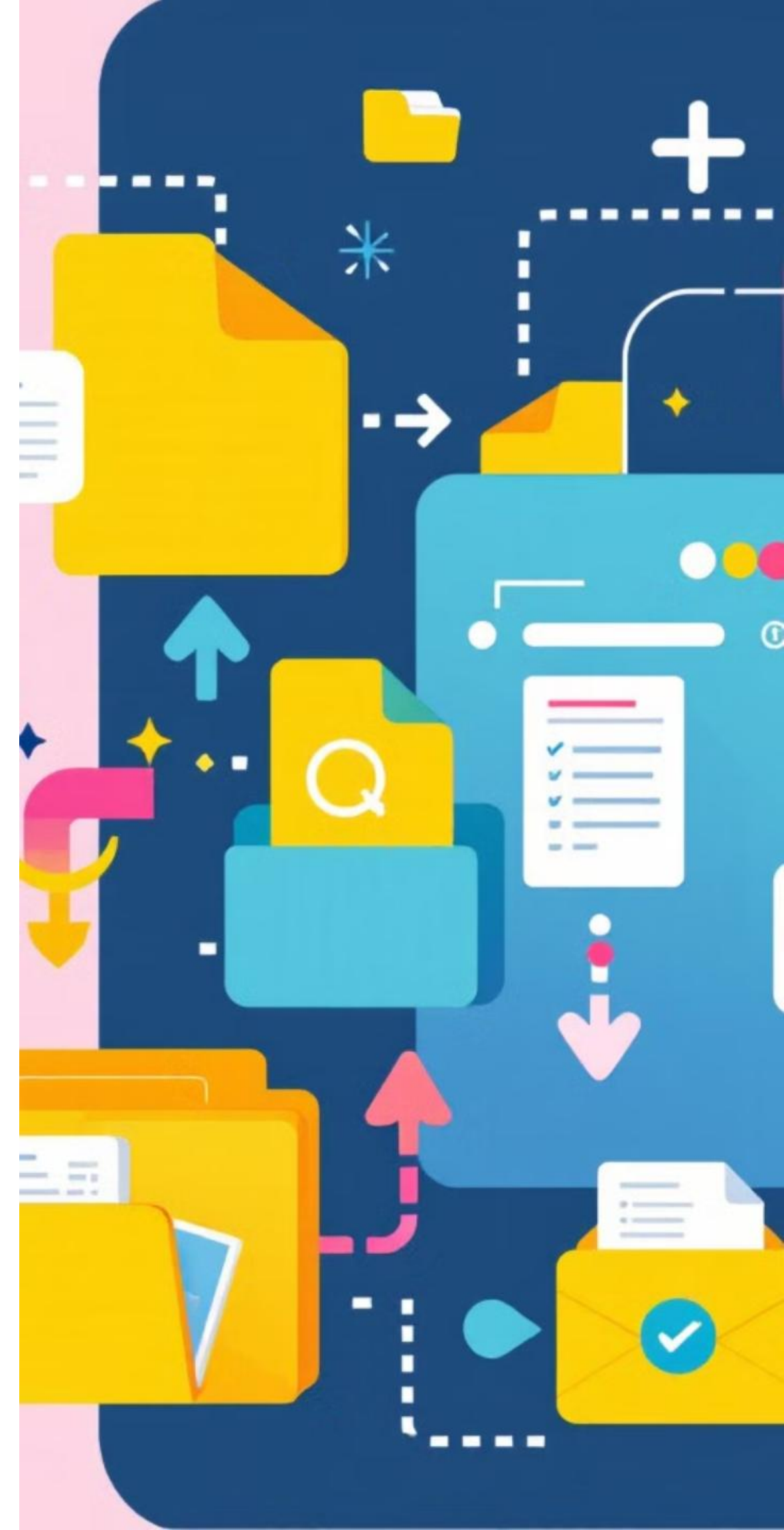
// ----- PARALELO: lanzar todos, esperar despues -----
t0 = System.nanoTime();
List<Process> procesos = new ArrayList<>();
for (int i = 0; i < N; i++) {
    procesos.add(tarea().start());    // bucle 1: solo lanzar
}
for (Process p : procesos) {
    p.waitFor();    // bucle 2: ahora si, esperar
}
long paralelo = (System.nanoTime() - t0) / 1_000_000;

System.out.printf("Secuencial: %d ms%n", secuencial);
System.out.printf("Paralelo:  %d ms%n", paralelo);
System.out.printf("Aceleracion: %.2f%n", (double) secuencial / paralelo);
}
}
```

7. DOCUMENTACIÓN DE LAS APLICACIONES



[INTELIJ: VIDEO](#)
[ECLIPSE: VIDEO](#)

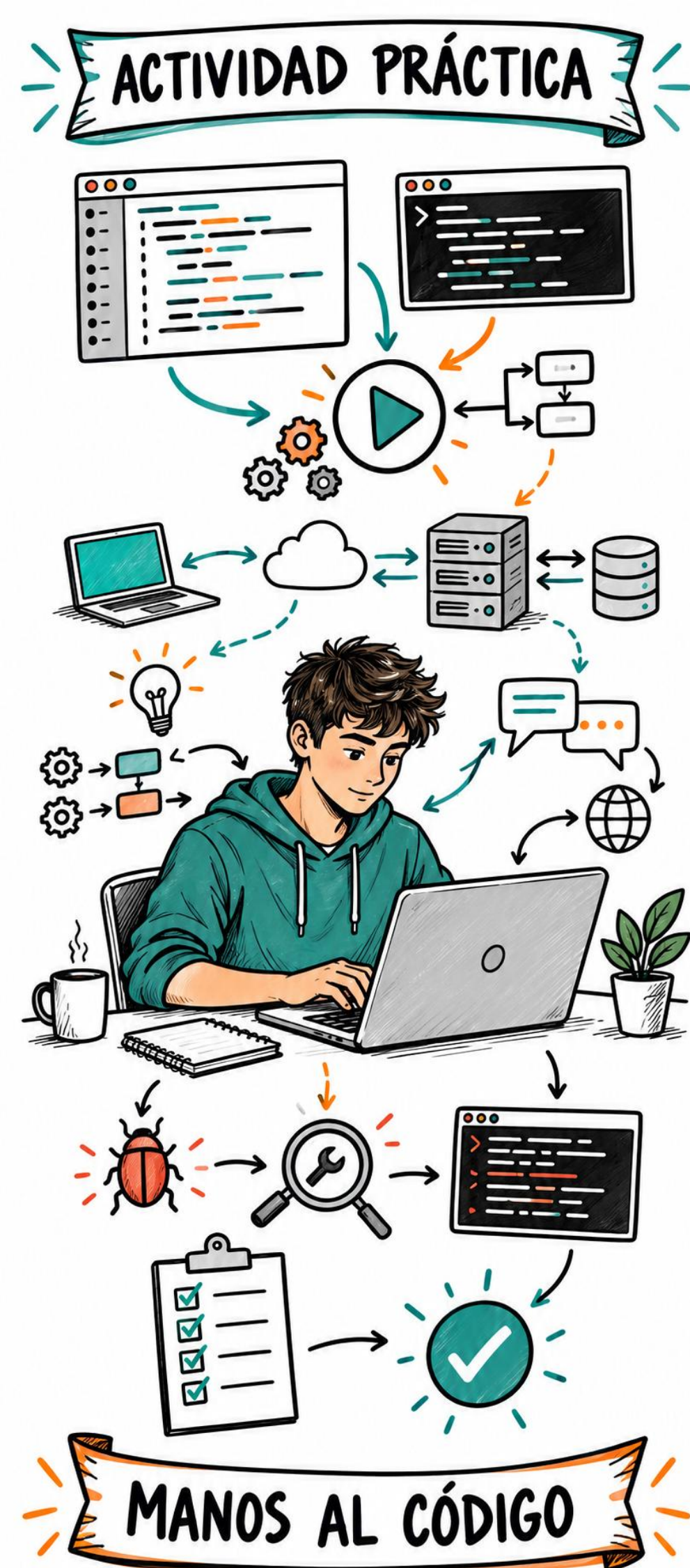




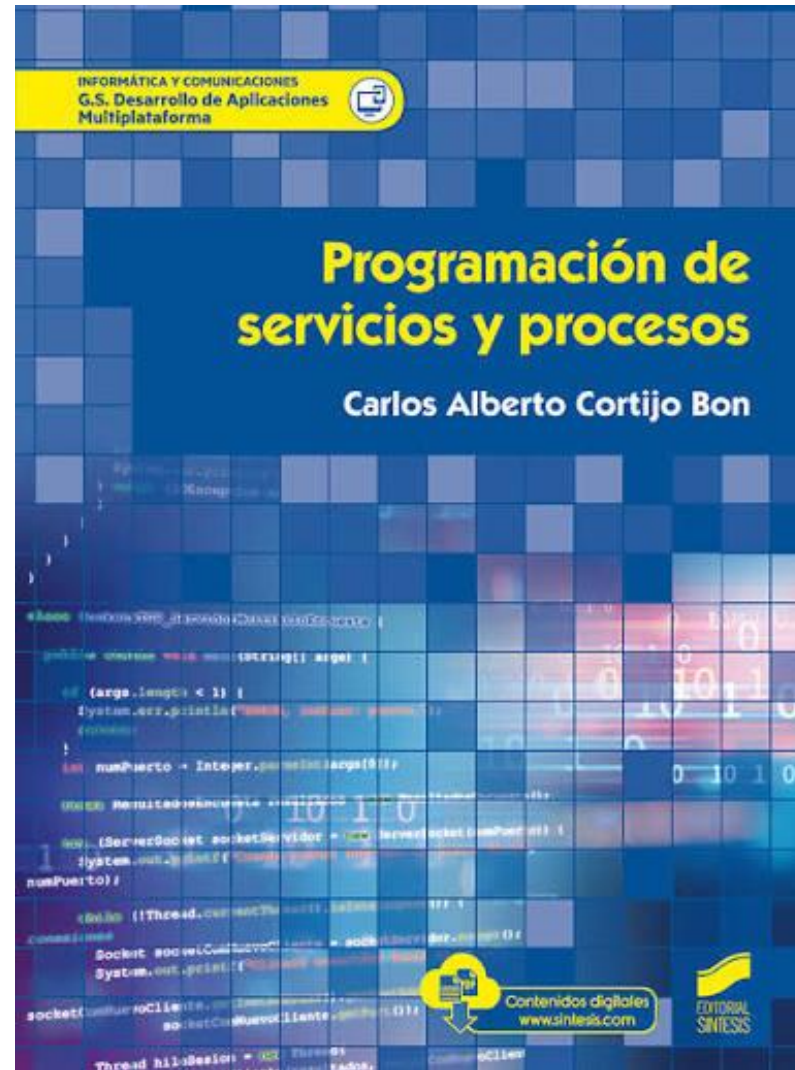
Video resumen (Aula Virtual)

Actividad RA1_A04

¿Compensa repartir el trabajo?
Medición y documentación



Bibliografía



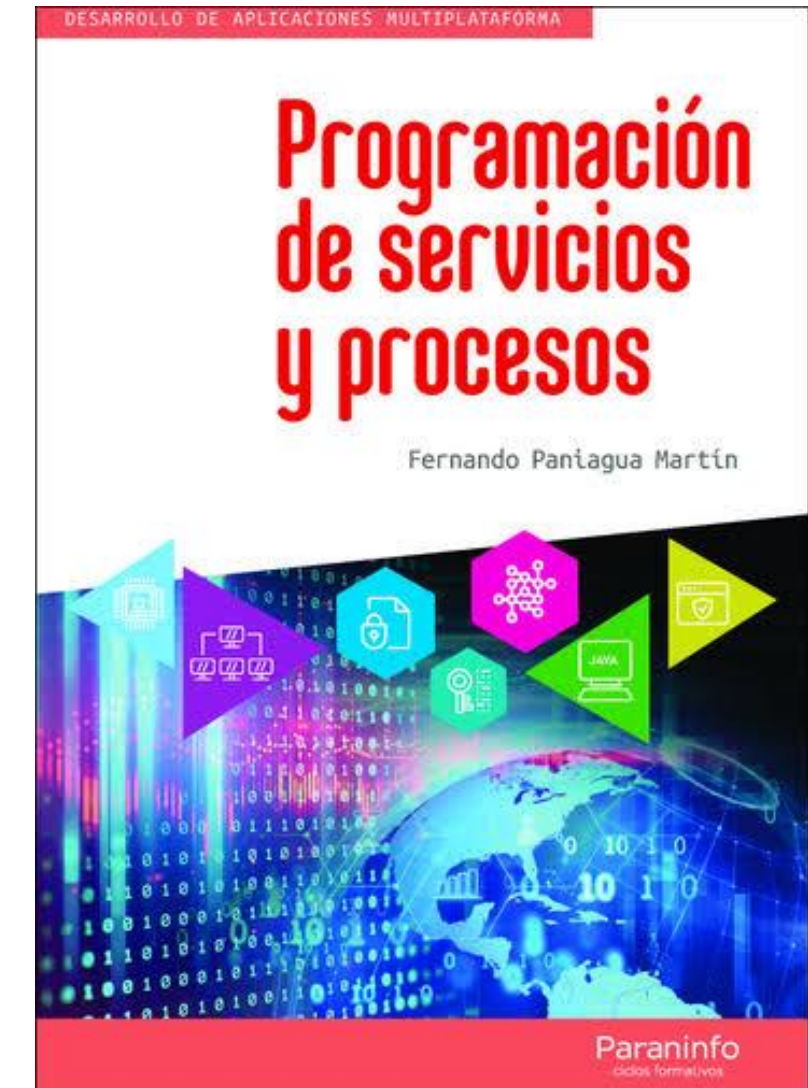
Cortijo Bon, Carlos A. (2021)

“Programación de servicios y procesos”. En editorial Síntesis (España). ISBN: 978-84-1357-104-1.



Córcoles Tendero, Jose Eduardo. (2014)

“Programación de servicios y procesos”. En editorial RA-MA (España). ISBN: 978-84-9964-391-5



Paniagua Martín, Fernando. (2022)

“Programación de servicios y procesos”. En editorial Paraninfo. ISBN 978-84-1366-526-9.